

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-aa4ac79fa5afc045f.jsonl`
- SHA-256: `4b5c90403603e8eabde069decb03eb2c698c81ce7e7c59dea167713ac00c325e`
- Source modified: `2026-06-08T08:43:41+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_d251ad6c-472`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-08T08:39:01.809Z)

Read the collector VENDOR/CVAT ingestion path and its stated PIPELINE ROLE.

Files: C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py (CVAT->canonical adapter), C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md, C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md, C:/Users/avidu/Projects/sports-data-collector/docs/DECISIONS.md, C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md, C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md, C:/Users/avidu/Projects/sports-data-collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv.

Determine, with anchors:

- The collector's CURRENTLY STATED role/positioning in the overall pipeline.
- The vendor (Roora) ingestion flow + intake manifest fields + how CVAT maps to canonical.
- How manual self-rating (the VLC rally-annotator) ingestion relates to or differs from the vendor flow today.
- Any documented stance on partial labels, golden-set, or regression.

2. assistant (2026-06-08T08:39:03.747Z)

I'll read the key documentation files to understand the collector's CVAT ingestion path, pipeline role, and related flows.

3. user (2026-06-08T08:39:03.992Z)

```
1      # Role in the rally-segmenter training pipeline
2
3      This repo is the data producer in a two-repo system. Its job is to curate
4      sports videos with per-rally `[start, end)` annotations and commercial-license
5      metadata, then hand that data to the badminton-highlight-indexer, which
6      processes, scores, and (Phase 4) trains an offline rally segmenter on it.
7
8      This doc explains why the `export` and `fetch` commands exist and how they fit
9      the bigger picture. The full decision records and end-to-end narrative live in the
10     indexer repo:
11
12     - `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
13     - `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
14       (acquisition policy), ADR-004 (offline learned segmenter).
15
16     ## Why this repo matters
17
18     The indexer's best rally mode refines local detections with a paid Gemini API.
19     The strategic goal is to replace that with an offline model trained on our own
20     commercially-licensed, annotated corpus ? making rally detection free, offline,
21     and self-improving. None of that is possible without a clean, licensable,
```

```

22 accurately-labelled, full-resolution dataset. That dataset is what this repo
23 produces.
24
25 ## What the two key commands do (and why)
26
27 - **`curate export`** ? emits the indexer's ground-truth format: per-video
28   `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29   Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30   table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31   `is_commercial`, so only licensable data flows
32   downstream. The annotations are **temporal-only** ? they train/evaluate rally
33   *segmentation*, not the frame-level shuttle detector (see ADR-002).
34
35 - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36   the DB **in place** so annotations and curation status are never lost. Uses
37   authenticated cookies for reliable, unthrottled downloads. The why and the
38   hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40 ## The one contract that matters: video identity
41
42 The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43 re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44 exact `local_path`, and any downstream processing must run on **that** file. This is
45 why acquisition (`fetch`) happens *before* the indexer processes anything.
46

```

4. user (2026-06-08T08:39:04.204Z)

```

1 # Rally Ingestion Pipeline ? End-to-End FAQ
2
3 How a vendor's rally annotations become training/eval ground truth, and how we
4 validate each delivery against our conventions. This is the operational companion
5 to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7 ```
8   vendor delivery                OUR side (collector)                modeling
9   (indexer)
10   ?????????????????? convert  ?????????????????? validate ??????????????????
11   ? CVAT export ? ????????????? ? canonical CSV ? ?????????? ? report .md ?
12   ? (boxes + attrs)? ? (per rally, ? ? report .json ?
13   ? OR ? ? ? decimal secs)? ? review .csv ???? human
14   verifies
15   ? canonical CSV ? ?????????????????? ??????????????????
16   flagged rallies
17   ?????????????????? ? ?
18   ? import-local (validated) ?
19   ? golden labels
20   ?????????????????? export ??????????????????
21   ? SQLite (rally ? ?????????? ? <id>.csv ?
22   start,end (indexer GT)
23   ? tables) ? ? manifest.json?
24   ?????????????????? ??????????????????
25   ?
26   slice clips / calibrate /
27   train
28   ```
29
30 ## The four commands (in order)
31
32 ```bash
33 # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
34 CSV)
35 python -m src.cli.curate convert-cvat --input job.xml \
36   --video-path match.mp4 --out canonical.csv --issues issues.txt
37

```

```

32 # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33 python -m src.cli.curate validate-delivery --input job.xml \
34     --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35 # ...or validate a CSV directly:
36 python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
37
38 # 3. After human review, import the (corrected) CSV into the DB
39 python -m src.cli.curate import-local --sport badminton \
40     --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42 # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43 python -m src.cli.curate export --out-dir data/eval_export
44 ```
45
46 ---
47
48 ## FAQ
49
50 ### What delivery formats do we accept?
51 Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
52 attributes) is normalised by `convert-cvat`; an already-canonical CSV is taken
53 as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
54 CSVs.
55
56 ### What is the "canonical" CSV?
57 One row per rally:
58 `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes`.
59 `start_time`/`end_time` are decimal seconds; `import-local` reads those.
60
61 ### How are timestamps handled? (the fps thing)
62 We recompute each rally's interval from its frame range × the true fps
63 (`seconds = frame / fps`). A vendor's CVAT time attributes are ignored ? a
64 step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
65 from `--fps` or `ffprobe` of `--video-path`. `mm:ss` is also accepted everywhere.
66
67 ### What does validation check?
68 | code | severity | meaning |
69 |---|---|---|
70 | `overlap` | blocker | two rallies' intervals overlap (`end[N] > start[N+1]`) |
71 | `zero_duration` | blocker | `end <= start` (single-sample / stray box) |
72 | `frame_out_of_range` | high | a box frame is past the video's last frame |
73 | `shot_density` | medium | long rally with implausibly few shots ? likely an inflated
span (often hides a merged rally) |
74 | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
for an omitted rally |
75 | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
76 | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
77
78 ### Blocker vs. review ? what's the gate?
79 `validate-delivery` exits non-zero if any blocker exists (so it can gate a bulk
80 loop / CI). Non-blocker issues don't stop the gate but populate the review sheet
81 for the human pass. `import-local` independently rejects overlaps/zero-duration, so
82 bad ground truth can't slip into the DB.
83
84 ### How are missed rallies caught? (the worst error)
85 Two complementary checks, because a miss shows up two ways:
86 1. Clean omission (a real hole in the timeline) ? `possible_missed_rally` (gap
heuristic).
87 2. Merged/inflated (the missed rally got swallowed into a neighbour's box, leaving
88 no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
89 the second rally inside it.
90 Neither is a guarantee ? they surface regions to review, they don't certify
91 completeness. (A future enhancement: a motion second-opinion over the raw video.)

```

```

92
93   ### What's the review sheet?
94   `<id>_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
95   blank `VERIFIED` / `true_start_mmss` / `true_end_mmss` / `true_ending_reason` /
96   `true_last_shot_outcome` / `reviewer_comment` columns. Open it next to the video
97   (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
98   a delivery into trusted golden labels.
99
100  ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
101  They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
102  unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
103  last shot went (in / net / out / n_a). A shuttle that flew long is
104  `forced_error/unforced_error` + `out` ? both kept, no information lost.
105
106  ### How do I onboard a brand-new video?
107  1. Get fps into the intake manifest (we provide it to the vendor).
108  2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
109     sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
110  3. The same suite runs for every video, so quality stays consistent at scale.
111
112  ### How do I pull a single rally out to eyeball it?
113  The slicer (indexer):
114  ```bash
115  python -m backend.tools.rally_slicer --video match.mp4 \
116      --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
117  ```
118  Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
119
120  ### Does coarse frame sampling hurt boundaries?
121  Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122  padding, so we don't flag sampling density.
123
124  ### Will a non-commercial license block export?
125  The video is **kept** in the DB regardless of license, but the default `export` is
126  **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
127  from the eval/training set unless you pass `--include-noncommercial`. So if a video
128  "goes missing" downstream, check its `is_commercial` flag first. (The observability
129  report raises a `license_noncommercial` warning for exactly this case.)
130
131  ### What is the observability report? (`*.report.json` / `*.report.md` / `*.summary.md`)
132  Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
133  per-video **observability report** (next to the video for `import-local`; into the
134  `--out-dir` / next to the canonical CSV otherwise):
135  - **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff
136    two runs to see what changed between deliveries.
137  - **`<id>.report.md`** ? the **technical** report: parse / validation / registration
138    stages plus **findings** (one per delivery issue, each citing this FAQ). Read this
139    to debug a delivery.
140  - **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the
141    rally count and coverage plus the plain-language list of rallies to review ? no
142    internal codes or severities.
143  This is in addition to the existing `<id>_report.md` / `_report.json` / `_review.csv`
144  (the human review sheet), which are unchanged.
145

```

5. user (2026-06-08T08:39:04.603Z)

```

1   """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3   Why this exists
4   -----
5   Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6   per-rally **CSV** that imports directly via `curate import-local`. A vendor may
7   instead do the work in CVAT and hand back the *CVAT* export: per-frame bounding
8   boxes, with the rally fields (`rally_number`, `shots_count`,

```

```

9     ``ending_reason`` ...) carried as CVAT **object attributes** on each box. The box
10    geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11    metadata to.
12
13    This module is the ingestion-side adapter that absorbs that vendor format so the
14    modeling repo never has to. It does two things, both deliberately on *our* side
15    so we fix mechanical problems rather than renegotiating them:
16
17    1. **Recompute time from the frame range.** A rally's interval is taken from the
18    first/last *frame* a box for that rally appears on:
19    ``seconds = frame / fps`` at the video's true fps. We **ignore** any
20    ``start_time``/``end_time`` attributes the vendor wrote ? a CVAT job that
21    samples every ``step`` frames and assumes 30 fps yields systematically wrong
22    times (we have seen ``start_time == frame / (step * 30)`` , i.e. off by the
23    sampling step at the wrong fps). The frame index is reliable; the vendor's
24    seconds are not.
25
26    2. **Surface ? never silently fix ? content defects.** Overlapping rallies,
27    zero-duration rallies, ending_reason values outside the agreed set, and
28    too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29    rally boundary would poison ground truth, so that stays a human/vendor call.
30
31    Supported input: the single-file CVAT XML export, in either **image mode**
32    (``<image><box>...``) or **track/interpolation mode** (``<track><box frame=...>``).
33    The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34    later if a vendor sends only those.
35    """
36    from __future__ import annotations
37
38    import os
39    import xml.etree.ElementTree as ET
40    from collections import Counter
41    from dataclasses import dataclass, field
42    from typing import Any, Dict, List, Optional, Tuple
43
44    from src.core.parsing import safe_int
45    from src.core.downloader import probe_resolution_fps
46
47    # The agreed cross-sport ending_reason vocabulary for net-separated sports.
48    # Kept here as a soft check only (a warning, never a hard failure) so this
49    # adapter does not depend on the annotation-spec wording.
50    DEFAULT_ALLOWED_REASONS = {
51        "winner",
52        "forced_error",
53        "unforced_error",
54        "service_fault",
55        "let",
56        "other",
57    }
58
59    # The observational outcome of the last shot ? orthogonal to ending_reason
60    # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
61    DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
62
63    # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
64    # schema has used suffixed names like "score_before (A-B)"; we match leniently.
65    _ATTR_ALIASES = {
66        "rally_number": ("rally_number",),
67        "shots_count": ("shots_count",),
68        "ending_reason": ("ending_reason",),
69        "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
70    (in/net/out)"),
71        "score_before": ("score_before", "score_before (a-b)"),
72        "score_after": ("score_after", "score_after (a-b)"),
73        "notes": ("notes", "note"),

```

```

73     }
74
75     CANONICAL_COLUMNS = [
76         "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
77         "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
78         "notes",
79     ]
80
81     @dataclass
82     class RallyRecord:
83         """One rally recovered from the CVAT export, time re-derived from frames."""
84         rally_number: int
85         start_time: float          # seconds, = frame_min / fps
86         end_time: float            # seconds, = frame_max / fps (see padding note)
87         shots_count: Optional[int]
88         ending_reason: Optional[str]
89         last_shot_outcome: Optional[str]
90         score_before: Optional[str]
91         score_after: Optional[str]
92         notes: Optional[str]
93         frame_min: int
94         frame_max: int
95         n_boxes: int
96         vendor_start_time: Optional[str] = None    # the (discarded) attribute value
97         vendor_end_time: Optional[str] = None
98
99         @property
100         def duration(self) -> float:
101             return self.end_time - self.start_time
102
103
104     @dataclass
105     class Issue:
106         """A defect found in the delivery. ``blocker`` issues stop a clean import."""
107         severity: str    # 'blocker' | 'high' | 'medium' | 'low'
108         code: str
109         message: str
110         rally_number: Optional[int] = None
111
112
113     @dataclass
114     class ConversionResult:
115         records: List[RallyRecord]
116         issues: List[Issue]
117         fps: float
118         step: int
119         meta: Dict[str, Any] = field(default_factory=dict)
120
121         @property
122         def n_blockers(self) -> int:
123             return sum(1 for i in self.issues if i.severity == "blocker")
124
125
126     def _mmss(seconds: float) -> str:
127         """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
128         m = int(seconds // 60)
129         s = seconds - m * 60
130         return f"{m}:{s:06.3f}"
131
132
133     def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
134         for alias in _ATTR_ALIASES[canonical]:
135             if alias in attrs and attrs[alias] != "":
136                 return attrs[alias]

```

```

137         return None
138
139
140 def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
141     """Parse a CVAT XML export into (meta, boxes).
142
143     ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
144     entry per visible box, from either image-mode or track-mode exports.
145     ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
146     """
147     if not os.path.exists(xml_path):
148         raise FileNotFoundError(f"CVAT export not found: {xml_path}")
149
150     root = ET.parse(xml_path).getroot()
151     meta: Dict[str, Any] = {}
152
153     job = root.find("./job")
154     if job is not None:
155         stop = job.findtext("stop_frame")
156         if stop is not None and stop.strip().isdigit():
157             meta["stop_frame"] = int(stop.strip())
158         ff = job.findtext("frame_filter") or ""
159         if "step=" in ff:
160             try:
161                 meta["step"] = int(ff.split("step=")[1].split())[0])
162             except (ValueError, IndexError):
163                 pass
164
165     def _attrs(el: ET.Element) -> Dict[str, str]:
166         out: Dict[str, str] = {}
167         for a in el.findall("attribute"):
168             out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
169         return out
170
171     boxes: List[Dict[str, Any]] = []
172
173     # Image mode: <image id=frame> ... <box>...</box>
174     for img in root.findall("image"):
175         try:
176             frame = int(img.get("id"))
177         except (TypeError, ValueError):
178             continue
179         for box in img.findall("box"):
180             boxes.append({"frame": frame, "attrs": _attrs(box)})
181
182     # Track mode: <track> ... <box frame=N outside=0>...</box>
183     for track in root.findall("track"):
184         for box in track.findall("box"):
185             if box.get("outside") == "1":
186                 continue # a track's terminating keyframe carries no real label
187             try:
188                 frame = int(box.get("frame"))
189             except (TypeError, ValueError):
190                 continue
191             boxes.append({"frame": frame, "attrs": _attrs(box)})
192
193     return meta, boxes
194
195
196 def _infer_step(frames: List[int]) -> int:
197     """Most common positive gap between consecutive distinct sampled frames."""
198     uniq = sorted(set(frames))
199     gaps = [b - a for a, b in zip(uniq, uniq[1:])] if b - a > 0]
200     if not gaps:
201         return 1

```

```

202         return Counter(gaps).most_common(1)[0][0]
203
204
205     def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
206         """Group boxes by rally_number and derive each rally's frame range -> seconds."""
207         if fps is None or fps <= 0:
208             raise ValueError("fps must be a positive number to convert frames to seconds")
209
210         by_rally: Dict[int, List[Dict[str, Any]]] = {}
211         for b in boxes:
212             rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
213             if rn is None:
214                 continue
215             by_rally.setdefault(rn, []).append(b)
216
217         records: List[RallyRecord] = []
218         for rn in sorted(by_rally):
219             items = by_rally[rn]
220             frames = sorted(b["frame"] for b in items)
221             attrs = items[0]["attrs"]
222             fmin, fmax = frames[0], frames[-1]
223             records.append(RallyRecord(
224                 rally_number=rn,
225                 start_time=fmin / fps,
226                 end_time=fmax / fps,
227                 shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
228                 ending_reason=_attr_lookup(attrs, "ending_reason"),
229                 last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
230                 score_before=_attr_lookup(attrs, "score_before"),
231                 score_after=_attr_lookup(attrs, "score_after"),
232                 notes=_attr_lookup(attrs, "notes"),
233                 frame_min=fmin,
234                 frame_max=fmax,
235                 n_boxes=len(frames),
236                 vendor_start_time=attrs.get("start_time"),
237                 vendor_end_time=attrs.get("end_time"),
238             ))
239         return records
240
241
242     def detect_issues(records: List[RallyRecord],
243                      stop_frame: Optional[int] = None,
244                      allowed_reasons: Optional[set] = None,
245                      allowed_outcomes: Optional[set] = None,
246                      max_gap_s: float = 15.0) -> List[Issue]:
247         """Find the defects that block or degrade a clean ground-truth import.
248
249         Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
250         slack of ~0.5s is acceptable for this pipeline because the downstream
251         highlight compiler pads every clip on output (badminton-highlight-indexer,
252         backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
253         absorbs the sampling uncertainty when highlights are trimmed.
254         """
255         allowed = allowed_reasons if allowed_reasons is not None else
256         DEFAULT_ALLOWED_REASONS
257         allowed_out = allowed_outcomes if allowed_outcomes is not None else
258         DEFAULT_ALLOWED_OUTCOMES
259         issues: List[Issue] = []
260
261         ordered = sorted(records, key=lambda r: r.rally_number)
262         for i, r in enumerate(ordered):
263             if r.end_time <= r.start_time:
264                 issues.append(Issue(
265                     "blocker", "zero_duration",
266                     f"Rally {r.rally_number}: end <= start "

```



```

265         f"(frames {r.frame_min}-{r.frame_max}); single-sample or stray box.",
266         r.rally_number))
267
268     if r.ending_reason is not None and r.ending_reason not in allowed:
269         issues.append(Issue(
270             "medium", "ending_reason_vocab",
271             f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
272             f"not in the agreed set {sorted(allowed)}.",
273             r.rally_number))
274
275     if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
276         issues.append(Issue(
277             "low", "outcome_vocab",
278             f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
279             f"not in {sorted(allowed_out)}.",
280             r.rally_number))
281
282     # Inflated span: a long rally with implausibly few shots usually means a
283     # stray/duplicated box stretched the frame range past the real ending.
284     if r.shots_count and r.duration >= 5.0:
285         sps = r.shots_count / r.duration
286         if sps < 0.3:
287             issues.append(Issue(
288                 "medium", "shot_density",
289                 f"Rally {r.rally_number}: {r.shots_count} shots over "
290                 f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
291                 f"may be inflated by a stray box.",
292                 r.rally_number))
293
294     if stop_frame is not None and r.frame_max > stop_frame:
295         issues.append(Issue(
296             "high", "frame_out_of_range",
297             f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
298             f"last frame {stop_frame}.",
299             r.rally_number))
300
301     if i > 0:
302         prev = ordered[i - 1]
303         if r.start_time < prev.end_time:
304             issues.append(Issue(
305                 "blocker", "overlap",
306                 f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
307                 f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
308                 f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
309                 r.rally_number))
310             elif r.start_time - prev.end_time > max_gap_s:
311                 # A long unlabeled stretch between two rallies is a cheap, label-only
312                 # second opinion for the worst error (a missed rally). It won't catch a
313                 # rally MERGED into its neighbour's span (shot_density/inflated catches
314                 # that), but it does catch an omitted rally that left a visible hole.
315                 gap = r.start_time - prev.end_time
316                 issues.append(Issue(
317                     "medium", "possible_missed_rally",
318                     f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
319                     f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
320                     f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
321                     gap.",
322                     r.rally_number))
323
324     return issues
325
326 def convert(xml_path: str,
327             fps: Optional[float] = None,
328             video_path: Optional[str] = None,
329             step: Optional[int] = None,

```

```

329         allowed_reasons: Optional[set] = None) -> ConversionResult:
330     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
331
332     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
333     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
334     that silent guess is exactly the bug this adapter exists to undo.
335     """
336     meta, boxes = parse_cvat_boxes(xml_path)
337
338     if fps is None and video_path:
339         _, _, probed = probe_resolution_fps(video_path)
340         fps = probed
341     if fps is None or fps <= 0:
342         raise ValueError(
343             "Could not determine fps. Pass fps=... or a readable video_path so "
344             "frames can be converted to seconds (seconds = frame / fps).")
345
346     if step is None:
347         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
348
349     records = build_records(boxes, fps)
350     issues = detect_issues(records,
351                             stop_frame=meta.get("stop_frame"),
352                             allowed_reasons=allowed_reasons)
353     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
354                             meta=meta)
355
356 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
357     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
358     import csv
359     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
360     with open(out_path, "w", newline="", encoding="utf-8") as f:
361         w = csv.writer(f)
362         w.writerow(CANONICAL_COLUMNS)
363         for r in sorted(records, key=lambda x: x.rally_number):
364             w.writerow([
365                 r.rally_number,
366                 _mmss(r.start_time), _mmss(r.end_time),
367                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
368                 r.shots_count if r.shots_count is not None else "",
369                 r.ending_reason or "",
370                 r.last_shot_outcome or "",
371                 r.score_before or "", r.score_after or "", r.notes or "",
372             ])
373
374
375 def render_issue_report(result: ConversionResult, source: str) -> str:
376     """A human-readable issue report (for review / sending back to the vendor)."""
377     lines = [
378         f"CVAT rally conversion report ? {source}",
379         f"  fps used           : {result.fps:.3f}",
380         f"  sampling step       : {result.step} frames (~{result.step / result.fps:.2f}s)",
381         f"  rallies parsed      : {len(result.records)}",
382         f"  issues               : {len(result.issues)} ({result.n_blockers} blocker)",
383         "",
384     ]
385     if not result.issues:
386         lines.append("No issues detected.")
387     else:
388         for sev in ("blocker", "high", "medium", "low"):
389             group = [i for i in result.issues if i.severity == sev]
390             if not group:
391                 continue
392             lines.append(f"[{sev.upper()}]")

```

```

393         for i in group:
394             lines.append(f" - ({i.code}) {i.message}")
395             lines.append("")
396     return "\n".join(lines)
397
398
399 # ----- #
400 # Validation reporting ? used by `curate validate-delivery`. #
401 # These turn a (records, issues) pair into the artifacts a human reviews: #
402 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
403 # ----- #
404
405 def _parse_secs(value: Optional[str]) -> Optional[float]:
406     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
407     s = (value or "").strip()
408     if not s:
409         return None
410     if ":" in s:
411         try:
412             total = 0.0
413             for part in s.split(":"):
414                 total = total * 60 + float(part)
415             return total
416         except ValueError:
417             return None
418     try:
419         return float(s)
420     except ValueError:
421         return None
422
423
424 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
425     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
426     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
427     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
428     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
429     """
430     import csv as _csv
431     if not os.path.exists(csv_path):
432         raise FileNotFoundError(f"CSV not found: {csv_path}")
433     records: List[RallyRecord] = []
434     with open(csv_path, newline="", encoding="utf-8") as f:
435         reader = _csv.DictReader(f)
436         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
437 [])]
438         for raw in reader:
439             row = {(k or "").strip().lower(): v for k, v in raw.items()}
440             rn = safe_int(row.get("rally_number"), None)
441             if rn is None:
442                 continue
443             start = _parse_secs(row.get("start_time"))
444             if start is None:
445                 start = _parse_secs(row.get("start_mmss"))
446             end = _parse_secs(row.get("end_time"))
447             if end is None:
448                 end = _parse_secs(row.get("end_mmss"))
449             if start is None or end is None:
450                 continue
451             records.append(RallyRecord(
452                 rally_number=rn, start_time=start, end_time=end,
453                 shots_count=safe_int(row.get("shots_count"), None),
454                 ending_reason=(row.get("ending_reason") or "").strip() or None,
455                 last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
456                 score_before=(row.get("score_before") or "").strip() or None,
457                 score_after=(row.get("score_after") or "").strip() or None,

```

```

457         notes=(row.get("notes") or "").strip() or None,
458         frame_min=-1, frame_max=-1, n_boxes=0,
459     ))
460     return records
461
462
463     def build_summary(records: List[RallyRecord], issues: List[Issue],
464                      source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
465         """Machine-readable validation summary (JSON / CI)."""
466         by_sev = {s: [i for i in issues if i.severity == s]
467                  for s in ("blocker", "high", "medium", "low")}
468         flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
469         return {
470             "source": source,
471             "fps": fps,
472             "n_rallies": len(records),
473             "n_issues": len(issues),
474             "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
475             "n_blockers": len(by_sev["blocker"]),
476             "passed": len(by_sev["blocker"]) == 0,
477             "flagged_rallies": flagged,
478             "issues": [{"severity": i.severity, "code": i.code,
479                        "rally_number": i.rally_number, "message": i.message} for i in
480             issues],
481         }
482
483     def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
484     -> str:
485         """Human validation report: scoreboard + issues by severity + rallies to review."""
486         n = len(records)
487         by_sev = {s: [i for i in issues if i.severity == s]
488                  for s in ("blocker", "high", "medium", "low")}
489         flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
490         clean = n - len(flagged)
491         lines = [
492             f"# Delivery validation ? {source}",
493             "",
494             f"- rallies: **{n}**",
495             f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
496             f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
497             {len(by_sev['low'])})",
498             f"- structurally-clean rallies: **{clean}/{n}**",
499             f"+ (f" ({100 * clean // n}%" if n else ""),
500             f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
501             "",
502         ]
503         if issues:
504             for sev in ("blocker", "high", "medium", "low"):
505                 grp = by_sev[sev]
506                 if not grp:
507                     continue
508                 lines.append(f"## {sev.upper()}")
509                 lines += [f"- ({i.code}) {i.message}" for i in grp]
510                 lines.append("")
511             else:
512                 lines.append("No issues detected. ?\n")
513         if flagged:
514             lines.append(f"## Rallies to review: {flagged}")
515         return "\n".join(lines)
516
517     def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
518     None:
519         """Auto-generate the annotatable manual-verification sheet (blank `true_` cols)."""

```

```

518     import csv
519     flags: Dict[int, List[str]] = {}
520     for i in issues:
521         if i.rally_number is not None:
522             flags.setdefault(i.rally_number, []).append(i.code)
523     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
524     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
525            "ending_reason", "last_shot_outcome", "auto_flags",
526            "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
527            "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
528     with open(out_path, "w", newline="", encoding="utf-8") as f:
529         w = csv.writer(f)
530         w.writerow(cols)
531         for r in sorted(records, key=lambda x: x.rally_number):
532             w.writerow([
533                 r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
534                 f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
535                 r.ending_reason or "", r.last_shot_outcome or "",
536                 ";".join(flags.get(r.rally_number, [])),
537                 "", "", "", "", "", "",
538             ])
539 
```

6. user (2026-06-08T08:39:04.669Z)

```

1     # Decision records
2
3     Lightweight ADRs for this repo. (The cross-repo training-pipeline ADRs live in
4     the indexer repo's `docs/DECISIONS.md`; this file is for collector-local choices.)
5
6     ---
7
8     ## ADR-001 ? Keep explicit per-sport code instead of a sport registry (for now)
9
10    **Status:** Accepted . **Date:** 2026-06-02
11
12    ### Context
13    Supporting a sport currently requires parallel edits across several sites:
14    - `SEGMENT_TABLES` + a `CREATE TABLE` + `insert_*` / `get_*` in `src/core/db.py`
15    - a `validate_*` method in `src/core/validator.py`
16    - `import-local` parse/validate/insert branches and interactive review/play in
17    `src/cli/curate.py`
18    - a schema in `src/core/schemas/`
19
20    A code review flagged this as copy-paste duplication and proposed a registry/plugin
21    mechanism (e.g. `SPORT_REGISTRY: dict[SportType, SportSpec]` driving generic
22    insert/get/validate/dispatch) to make adding a sport a single entry.
23
24    ### Decision
25    **We are keeping the explicit per-sport code and NOT introducing a registry
26    abstraction at this time.**
27
28    ### Rationale
29    - **It's well-tested and mechanical.** Coverage is ~87%; adding a sport is a known,
30    ~30-minute "mirror an existing sport" task (checklist below), not a research problem.
31    - **Schemas are heterogeneous.** Rally sports (badminton, pickleball, table tennis)
32    share `rally_number` / `shots_count` / `ending_reason`, but tennis (`set_score`,
33    `game_score`, `server`, `ending_type`) and cricket (`over`, `ball`, `runs`,
34    `wicket`, `extras`) differ enough that a single generic table/row mapper would leak
35    special-cases ? trading visible duplication for hidden complexity.
36    - **Explicit code is greppable.** Contributors can follow one sport end-to-end without
37    learning a registry indirection. Premature abstraction is harder to undo than dup.
38    - **Low, slow growth.** Five sports today; new ones arrive rarely.
39
40    ### Consequences

```

```

40 - Adding a sport touches ~5 sites (see checklist). We accept that boilerplate.
41 - If the cost rises, ADR-001 should be revisited and superseded.
42
43 ### Revisit triggers (when to build the registry)
44 - Sport count grows beyond ~8, **or**
45 - We add a 4th+ rally-identical sport and the dispatch churn starts causing bugs, **or**
46 - A new cross-cutting feature would otherwise require editing every per-sport branch.
47
48 ### Future design sketch (when revisited)
49 ```
50 @dataclass
51 class SportSpec:
52     schema_class: type
53     table_name: str
54     pk_columns: tuple[str, ...]
55     columns: tuple[str, ...]
56     validate_fn: Callable
57
58     SPORT_REGISTRY: dict[SportType, SportSpec] = { ... }
59     # Generic: insert_segments(sport, items) / get_segments(sport, video_id) /
60     #         validate(sport, items) / import-local dispatch driven by the registry.
61     ```
62
63 ---
64
65 ## Checklist: adding a new sport today (under ADR-001)
66
67 1. `src/core/schemas/<sport>.py` ? define the segment model (mirror the closest existing
sport) with a `start_time < end_time` validator.
68 2. `src/core/db.py` ? add to `SEGMENT_TABLES`, add a `CREATE TABLE`, `insert_<sport>()`,
`get_<sport>()`, and a cascade `DELETE` in `delete_video`.
69 3. `src/core/validator.py` ? add `validate_<sport>()` (duration + no-overlap).
70 4. `src/cli/curate.py` ? add branches in `import_local` (validate + insert),
`_parse_local_annotations` (JSON **and** CSV), and the interactive review/play.
71 5. `config/config.yaml` ? add the sport to `supported_sports` (and confirm the
`SportType` enum value in `schemas/base.py`).
72 6. `tests/` ? mirror the existing db/validator/cli tests for the new sport.
73

```

7. assistant (2026-06-08T08:39:06.651Z)

Now let me read the remaining annotation and manifest documentation:

8. user (2026-06-08T08:39:07.136Z)

```

1 # Roora Annotation Brief ? Rally Golden Set
2
3 **Project:** racket-sport rally segmentation golden / regression dataset
4 **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
doubles**)
5 **Companion doc:** [ANNOTATION_GUIDE.md](./ANNOTATION_GUIDE.md)
6
7 > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8 > imports directly via `python -m src.cli.curate import-local`.
9
10 ## 0. What we need in one sentence
11 For each provided match video, produce **one CSV file** listing **every rally** with its
12 start time, end time, number of shots, and how it ended ? **excluding all dead time**.
13
14 ## 1. Deliverable format (canonical)
15 One CSV per video (UTF-8), named exactly `<video_id>.csv` (we supply the `video_id`).
16 One row per rally, chronological, header row required.
17
18 | column | meaning |

```

```

19 |---|---|
20 | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21 | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22 | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23 | `start_time` | decimal **seconds** (e.g. `72.40`). **Auto-filled** from `start_mmss`.
|
24 | `end_time` | decimal **seconds**. **Auto-filled** from `end_mmss`. |
25 | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26 | `ending_reason` | one of: `winner` \| `forced_error` \| `unforced_error` \|
`service_fault` \| `let` \| `other` (**why** it ended) |
27 | `last_shot_outcome` | one of: `in` \| `net` \| `out` \| `n_a` (**where** the last shot
landed; `n_a` for service_fault/let) |
28 | `score_before` | optional, `"A-B"`. Blank if not confidently readable. |
29 | `score_after` | optional, `"A-B"`. |
30 | `notes` | optional free text. **Required** when `ending_reason = other`. |
31
32 **Times (mm:ss ? decimal seconds).** Raters type only `start_mmss`/`end_mmss`. The
template
33 Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
row 2):
34
35 ```
36 =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
37 ```
38
39 On **Download as CSV**, the computed decimal seconds are exported. (If your tool already
40 shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
columns blank.)
41
42 Example (badminton):
43 ```csv
44 rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
_outcome,score_before,score_after,notes
45 1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
46 2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
47 3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
pressure)
48 ```
49
50 **Critical format rules**
51 - `start_time`/`end_time` that reach us **must be decimal seconds** (the formula
guarantees this).
52 - Times are relative to the **start of the file we provide**. Do **not** trim, clip, or
53 re-encode the video ? our pipeline identifies a video by its file checksum.
54 - `end_time` > `start_time` for every row.
55 - No overlaps: `end_time[N]` <= `start_time[N+1]`.
56 - Everything between one rally's `end_time` and the next rally's `start_time` is **dead
time** and gets **no row**.
57
58 ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
59 1. **Ignore all dead time.** Only label active play.
60 2. **Include every rally ? exclude none.** Every served point, including **service
faults**,
61 1?2 shot rallies, and **lets/replays**. A missed rally is the worst error in this
project.
62
63 **Exception: warm-up / knock-up is NOT a rally** ? pre-match and between-games warm-
up looks
64 like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real
serve,
65 cooperative (not competitive) hitting, the score not progressing, often several
shuttles/balls
66 in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
it.
67
68 3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-

```

```

clips are not shots.
67
68     ## 3. Scope & phasing
69     **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
70 tennis), each ` $< 3$  min` with ~10?12 rallies. This is the **paid calibration round**; we review
71 jointly and
72 lock the Guide before scale-up.
73
74     **Phase B ? Scale (after a successful pilot):** ~12?15 videos **per sport** (~36?45
75 total), each
76 ~30 min. Same format, rules, and template.
77
78     > **Doubles are included** across the program. Rally rules are identical to singles; the
79 > badminton- and pickleball-specific doubles notes are in the Guide.
80
81     ## 4. Quality bar (acceptance criteria)
82     Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
83 then a
84 ~20% human spot-check. A file is returned for rework (in scope) if it misses:
85 - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
86 - **Boundaries:** `start_time`/`end_time` within  $\pm 0.3$  s** of true serve-contact /
87 landing.
88 - **shots_count:** exact for rallies ? 15 shots;  $\pm 1$  allowed for longer/faster rallies.
89 - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
90 - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
91
92     ## 5. Per-sport quick reference (full version + examples in the Guide)
93     **Badminton (singles & doubles)** ? start: racket?shuttle serve contact . end: shuttle
94 floors / fault . shot: each racket?shuttle contact.
95     **Pickleball (singles & doubles)** ? start: paddle?ball serve contact . end: double
96 bounce / out / net / fault . shot: each paddle?ball contact (floor bounces are **not** shots;
97 two-bounce rule is normal play).
98     **Table tennis** ? start: racket?ball serve strike (not the toss) . end: failed legal
99 return . shot: each racket?ball contact (table bounces are **not** shots). Net-cord serve =
100 `let`.
101
102     ## 6. ending_reason values (decision tree in the Guide)
103     This vocabulary attributes **why** the point ended; it is shared across all
104 net-separated sports (badminton, pickleball, table tennis, tennis). Record **where**
105 the last shot physically landed in the separate **`last_shot_outcome`** column
106 (`in` \ | `net` \ | `out` \ | `n_a`) ? e.g. a forced error that flew long is
107 `ending_reason=forced_error, last_shot_outcome=out`.
108
109     | value | meaning |
110     |---|---|
111     | `winner` | last shot landed **in** and the opponent made no legal contact (a clean
112 winner). |
113     | `forced_error` | the losing player erred (netted / out / missed) **under pressure** ?
114 stretched, wrong-footed, or rushed by the opponent's strong shot. |
115     | `unforced_error` | the losing player erred on a **routine, unpressured** ball they had
116 time and position to make. |
117     | `service_fault` | the serve itself failed / was illegal. |
118     | `let` | the point was replayed. |
119     | `other` | anything else / cannot tell. **Must** add a note. |
120
121     ## 7. Recommended capture method
122     Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
123 can
124 **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
125 tool
126 (Label Studio / CVAT). The deliverable is the CSV in the schema above.
127 `seconds = frame_number / fps` (we provide fps per video).
128
129     ## 8. Logistics & contacts

```



```

116 - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
117 - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[`INTAKE_MANIFEST_TEMPLATE.csv`](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
118 - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed
`<video_id>.csv` files.
119 - Blank annotation template Sheet (mm:ss ? seconds formula from $1 pre-filled):
**TODO(avidullu): create it and paste the link.**
120 - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
questions.
121 - NDA before any footage is shared ? template [`NDA_TEMPLATE.md`](./NDA_TEMPLATE.md).
**TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
122 - Roora contact on file: `info@rooragr.com` (HSR Layout, Bangalore).
123
124 ## 9. Pricing request (please quote)
125 1. Price **per rally** and **per minute** of footage.
126 2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127 3. Indicative **scale** price for ~36?45 longer videos.
128 4. Turnaround for the pilot and throughput (rallies/day) at scale.
129 5. QA process and rework policy.
130 6. Whether annotators have racket-sport familiarity (helpful, not required).
131 7. Confirmation you can deliver the CSV schema above.
132

```

9. user (2026-06-08T08:39:07.773Z)

```

1 # Illustrated Annotation Guide ? Rally Labeling
2
3 Supplement to [`ROORA_BRIEF.md`](./ROORA_BRIEF.md). Sports: **Badminton, Pickleball,
Table Tennis** (singles **and** doubles).
4
5 > **Illustrations:** diagrams are drawn in text so they survive copy/paste. Boxes marked
6 > `[SCREENSHOT: ...]` are placeholders.
7 > **TODO(avidullu): replace every `[SCREENSHOT: ...]` placeholder with a real frame grab
8 > from the pilot videos before sending this guide to Roora.**
9
10 ## Part 1 ? What you are producing
11 One CSV per video (from the template Sheet), one row per rally:
12
13 ```
14 rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes
15 ```
16
17 You type: `rally_number`, `start_mmss`, `end_mmss`, `shots_count`, `ending_reason`,
18 `last_shot_outcome` (+ optional score/notes). The `start_time`/`end_time` decimal-second
columns
19 **fill themselves** via a formula already in the template.
20
21 For every rally you record:
22 - **When it started** (serve contact) ? `start_mmss` (`m:ss.mmm`, e.g. `1:12.40`)
23 - **When it ended** (shuttle/ball dead) ? `end_mmss`
24 - **How many shots** happened ? `shots_count`
25 - **Why it ended** ? `ending_reason` . **Where the last shot landed** ?
`last_shot_outcome`
26
27 Everything else (gaps between rallies) you **ignore**.
28
29 ## Part 2 ? The three golden rules
30
31 ### Rule 1: Ignore all dead time
32 A rally is **continuous active play**. The moment play stops, the rally is over. The
time
33 until the next serve is **dead time** and is never labeled.

```

```

34
35   ...
36   DEAD          RALLY 1          DEAD          RALLY 2          DEAD
37   (ignore) |=====| (ignore) |=====| (ignore)
38           ^               ^               ^               ^
39           serve          shuttle          serve          shuttle
40           contact        lands           contact        lands
41           =start_1       =end_1          =start_2       =end_2
42   ...
43
44   Dead time includes: walking back to serve, picking up the shuttle/ball, wiping sweat,
45   bouncing the ball before serving, the score being announced, replays/slow-mo, crowd or
coach
46   shots, changeovers, towel/medical timeouts, and **pre-match / between-games warm-up
47   (knock-up)** ? see the Rule 2 caveat below.
48
49   ### Rule 2: Include every rally ? even the ugly ones
50   Every served point gets a row, including:
51   - **Service faults** (serve into net/out/illegal) ? usually `shots_count = 1`,
`ending_reason = service_fault`.
52   - **Very short rallies** (serve + one error).
53   - **Lets / replays** ? `ending_reason = let`; the replayed point is its **own** next
row.
54
55   A **missed rally is the worst error** in this project ? the model must learn that short
and
56   failed rallies are still rallies.
57
58   > **?? BUT warm-up / knock-up is NOT a rally** ? a common trap, because Rule 2 says
"include
59   > everything." Pre-match and between-games warm-up *looks* like rallying but is not
scored play;
60   > **do not label it.** How to tell it apart:
61   > - **No real serve.** Real play starts with a proper serve from the service position
(diagonal,
62   >   behind the service line). Warm-up has players feeding cooperatively from mid-court
with no
63   >   serve ? not a rally.
64   > - **Cooperative, not competitive.** Gentle hits straight to each other; nobody is
trying to win
65   >   the point (no smashes-to-win, no scrambling/diving).
66   > - **Score isn't progressing.** No umpire / scoreboard not running / players casually
retrieve
67   >   and re-feed; the score doesn't change. Often **several shuttles/balls** in use at
once.
68   > - **Per sport:** badminton & pickleball ? real play starts with the underhand serve
from the
69   >   service court / behind the baseline; table tennis ? real play starts with the
tossed,
70   >   two-bounce serve (knock-up is cooperative rallying without it).
71   >
72   > **When unsure:** find the first proper serve where the score starts to progress ?
labeling
73   > begins there. If you genuinely can't tell whether the match has started, **flag the
clip to us
74   > ? don't guess.**
75
76   ### Rule 3: A shot = one contact with the racket / paddle / bat
77   Count every time the shuttle/ball touches a racket, paddle, or bat. The serve is shot 1.
78   - **Count:** any racket/paddle/bat contact (serve, return, smash, block?).
79   - **Don't count:** floor bounces, table bounces, net clips, the toss before a serve.
80
81   5-shot example: `serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5)` ? `shots_count =
5`, `ending_reason = winner`.
82

```

```

83     ## Part 3 ? Where exactly does a rally start and end?
84     **START (`start_mmss`):** the exact frame the server's racket/paddle/bat **contacts**
the
85     shuttle/ball. Not the toss, not the wind-up ? the contact frame.
86     `[SCREENSHOT: serve-contact frame]`
87
88     **END (`end_mmss`):** the exact frame the shuttle/ball becomes **dead**:
89     - Badminton: shuttle touches the floor, or play stops on a fault.
90     - Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.
91     - Table tennis: the second bounce on one side, a non-serve net hit, or off the table.
92
93     `[SCREENSHOT: end frame]`
94
95     If occluded/off-screen, pick the closest clear frame and add a note.
96     **Boundary tolerance:  $\pm 0.3$  s** ( $\sim \pm 9$  frames at 30 fps).
97
98     ## Part 4 ? ending_reason: how to choose
99     Pick exactly **one** value for how the rally ended. Stop at the first match:
100
101     1. Did the **serve** itself fail / was illegal (and ended the point)? ? `service_fault`
102     2. Was the point **replayed** (let / interruption)? ? `let`
103     3. Did the final shot land **in** and the opponent make **no legal contact** ? ? `winner`
(a clean winner)
104     4. Did the losing player make an error (netted / out / missed) **under pressure** ?
105         stretched, wrong-footed, or rushed by a strong shot? ? `forced_error`
106     5. Did the losing player make an error on a **routine, unpressured** ball they had
107         the time and position to make? ? `unforced_error`
108     6. None of the above / cannot tell ? `other` (add a note)
109
110     **Forced vs unforced ? the one judgment call.** Decide using the opponent's *preceding*
111     shot:
112     - `forced_error` = the opponent's shot **caused** the miss (pace, placement, deception;
113         the player barely reached it or was pulled out of position).
114     - `unforced_error` = a clean, makeable ball the player missed on their own.
115     - When genuinely unsure between the two, choose `unforced_error` and add a brief note.
116
117     Notes:
118     - A clean `winner` means **no legal contact** by the opponent. If they got a racket /
paddle /
119         bat on it but netted or hit out, that is a `forced_error` / `unforced_error` (their
shot), not a `winner`.
120     - A serve into the net is `service_fault` (rule 1 beats everything).
121
122     ### Then set `last_shot_outcome` ? where the last shot physically went
123     This is a **separate, observational** column (no judgment): just look at the last shot.
124     - `in` ? landed in (winners; also a put-away the opponent couldn't reach).
125     - `net` ? went into the net.
126     - `out` ? landed outside the court / past the lines.
127     - `n_a` ? not applicable: use for `service_fault` and `let`.
128
129     So each rally gets both: **why** (`ending_reason`) and **where** (`last_shot_outcome`) ?
e.g. a
130     smash winner = `winner, in`; a clear forced long = `forced_error, out`; a routine ball
netted =
131     `unforced_error, net`.
132
133     ## Part 5 ? Per-sport detail
134
135     ### Badminton
136     - **Serve:** underhand, below the waist, diagonal. Start = racket?shuttle contact.
137     - **Rally end:** shuttle hits the floor, or play stops on a fault.
138     - **Shots:** every racket?shuttle contact; serve = shot 1.
139     - **Service faults** (1-shot rally, `service_fault`): serve into net, serve outside
service court, contact above waist, foot fault.
140

```

```

141     Worked example:
142     ```
143     [SCREENSHOT: badminton serve]
144     Rally 4: serve 1:28.30, long rally, smash winner, shuttle lands 1:44.10, 12 contacts.
145     4, 1:28.30, 1:44.10, (auto), (auto), 12, winner, 3-5, 4-5,
146     Rally 5: serve 1:57.00 straight into the net.
147     5, 1:57.00, 1:57.90, (auto), (auto), 1, service_fault, 4-5, 4-6,
148     ```
149
150     **Badminton ? DOUBLES (2 players per side).** All rally rules are **identical** to
    singles:
151     - `start` = serve contact; `end` = shuttle down / fault; a **shot = any racket?shuttle
    contact by _either_ player on a side**.
152     - `shots_count` = **total** contacts across both teams in the rally. **Do not track
    which player hit** ? just count every contact.
153     - Net exchanges (drives, flat pushes) come fast ? **frame-step** so you neither miss nor
    double-count contacts.
154     - Only the **designated receiver** returns the serve; if the wrong player returns, the
    rally ends on a fault ? still a row (`service_fault` if it's a serve/receiving-position fault,
    else `other` + note).
155     - Two teammates may **not** both hit the shuttle in succession (double-hit). If you see
    two contacts on the **same** side back-to-back, that's a fault ending the rally ? count both
    contacts, then the rally ends.
156
157     ### Pickleball
158     - **Serve:** underhand, paddle below waist, hit diagonally; must clear the non-volley
    zone (the "kitchen"). Start = paddle?ball contact.
159     - **Two-bounce rule:** the serve must bounce once in the receiver's court and the return
    must bounce too, before either side may volley. These bounces are **normal play**, not rally
    ends, and bounces are **not** shots.
160     - **Rally end:** ball bounces twice, lands out, hits the net (fails to cross), or a
    fault (e.g. volleying in the kitchen).
161     - **Shots:** every paddle?ball contact only; serve = shot 1; floor bounces are not
    shots.
162     - **Service faults** (`service_fault`): serve into net, out/long, short in the kitchen,
    foot fault.
163
164     Worked example:
165     ```
166     [SCREENSHOT: pickleball serve + kitchen line]
167     Rally 2: serve 0:33.10; serve bounces, return bounces, a few volleys; receiver nets an
    easy one at 0:41.85; 6 paddle contacts.
168     2, 0:33.10, 0:41.85, (auto), (auto), 6, unforced_error, 1-0, 2-0, easy ball netted
169     ```
170
171     **Pickleball ? DOUBLES (the standard format, 2 per side).** All rally rules
    **identical** to singles:
172     - `start` = serve contact; a **shot = any paddle?ball contact by _either_ teammate** ;
    floor bounces still not shots; two-bounce rule still applies.
173     - `shots_count` = **total** paddle contacts across both teams; don't track which player.
174     - **Serving sequence (FYI only ? you do NOT annotate it):** both partners serve before
    the serve passes to the opponents, except the very first service turn of the game. The called
    score has three numbers (server score ? receiver score ? server #1/#2). Ignore this for
    annotation; just mark boundaries / shots / ending.
175     - If both teammates contact the ball in succession, that's a fault ? rally ends; count
    the contacts, set the ending per the fault.
176
177     ### Table tennis
178     - **Serve:** ball tossed up from an open palm, then struck so it bounces once on each
    side. Start = the **racket?ball strike** (not the toss).
179     - **Rally end:** a player fails a legal return ? ball double-bounces on their side, goes
    off the table, or is hit into the net on a non-serve shot.
180     - **Shots:** every racket?ball contact only; serve = shot 1; table bounces are not
    shots.
181     - **Let:** a serve that clips the net but is otherwise legal ? the point is replayed.

```

```

`ending_reason` = let`, and the replay is its own next row.
182 - **Service faults** (`service_fault`): missed/illegal toss, serve into net, serve off
the table.
183 - **Speed warning:** contacts can be < 0.2 s apart ? **step frame-by-frame** to count
shots.
184 - **Doubles (if a clip appears):** partners must hit in **strict alternation** and serve
diagonally; a missed alternation is a fault that ends the rally. Same rule ? count **every**
racket?ball contact as a shot; `shots_count` is the team total.
185
186 Worked example:
187 ```
188 [SCREENSHOT: table-tennis serve contact]
189 Rally 9: serve 4:10.40; fast 7-contact rally; forehand winner, opponent no contact; dead
4:14.05.
190 9, 4:10.40, 4:14.05, (auto), (auto), 7, winner, 6-9, 6-10,
191 Rally 10: serve 4:22.10 clips the net (otherwise good) -> let.
192 10, 4:22.10, 4:23.00, (auto), (auto), 1, let, 6-10, 6-10, net-cord serve, replayed
193 ```
194
195 ## Part 6 ? Edge cases & FAQ
196 - **Two rallies with almost no gap?** Still two rows. `end_time[N] <= start_time[N+1]`,
never overlap.
197 - **Interrupted & replayed?** Label the interrupted attempt as its own row
(`ending_reason=let`), then the replayed point as the next row.
198 - **Serve tossed but caught / re-served (no contact)?** No contact = no rally yet. Start
at the actual serve contact; don't label the aborted toss.
199 - **Opponent reached the ball but netted/hit out?** That's a
`forced_error`/`unforced_error` (their shot), not `winner` ? `forced` if your shot pressured
them, `unforced` if it was routine.
200 - **Can't read the scoreboard?** Leave `score_before`/`score_after` blank. Never guess.
201 - **"Let" where the rules say play on?** If play continued, it's **not** a let ? keep
one rally and judge the real ending.
202 - **Warm-up / knock-up / practice points?** Not real rallies ? **do not label them.**
Start at the first real, scored serve. See the **Rule 2 caveat (Part 2)** for how to tell warm-
up from real play (no serve, cooperative not competitive, score not progressing, several
shuttles/balls in use). When unsure whether the match has started, flag the clip ? don't guess.
203 - **Doubles?** Same rules ? a shot is any one racket/paddle contact by whichever player
hits it. See the badminton and pickleball **DOUBLES** notes in Part 5; never attribute
`shots_count` to an individual, just total the contacts.
204 - **Server completely whiffs (no contact)?** Rare. True zero contact ? `shots_count` =
0, `ending_reason` = service_fault, add a note. A graze = 1 contact = 1 shot.
205
206 ## Part 7 ? Self-check before you submit (per video)
207 - [ ] `rally_number` is 1,2,3? no gaps or repeats.
208 - [ ] Every `start_time` < `end_time` (check the auto-filled decimal columns populated).
209 - [ ] No overlaps: `end_time[N] <= start_time[N+1]`.
210 - [ ] Times entered as `m:ss.mmm`; decimal `start_time`/`end_time` auto-filled and look
right.
211 - [ ] No dead-time rows (scan the gaps ? dead time, not missing rallies).
212 - [ ] Every served point present, including service faults and lets.
213 - [ ] `shots_count` looks right (re-step any rally you rushed; doubles = team total).
214 - [ ] `ending_reason` ?
{`winner`,`forced_error`,`unforced_error`,`service_fault`,`let`,`other`} ; notes filled when
`other`.
215 - [ ] `last_shot_outcome` ? {`in`,`net`,`out`,`n_a`} (`n_a` for service_fault/let).
216 - [ ] File downloaded as `.csv`, header row present, UTF-8.
217 - [ ] Did **not** trim / re-encode / rename the source video.
218
219 ## Part 8 ? Mini glossary
220 - **Rally** ? one continuous passage of play from serve contact to the shuttle/ball
dead.
221 - **Shot** ? one contact between the shuttle/ball and a racket/paddle/bat (either
teammate in doubles).
222 - **Dead time** ? any time the shuttle/ball is not in active play. Never labeled.
223 - **Service fault** ? a failed/illegal serve that ends the point. Still a labeled rally.

```

224 - **Let** ? a replayed point. Labeled as its own rally with `ending\_reason=let`.
225 - **Winner / forced_error / unforced_error** ? the final shot was a clean winner
(opponent made no legal contact) / the loser's error was forced by a pressuring shot / the
loser's error was on a routine, unpressured ball.
226 - **start_time / end_time** ? decimal seconds from the start of the provided file (auto-
filled from mm:ss).
227

10. user (2026-06-08T08:39:07.810Z)

```
1 video_id,sport,format,match_name,source_filename,drive_link,fps,duration_mmss,expected_r
allies,assigned_to,status,notes
2 pilot_badminton_01,badminton,singles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:45,10-12,,todo,deliver as pilot_badminton_01.csv
3 pilot_pickleball_01,pickleball,doubles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:30,10-12,,todo,deliver as pilot_pickleball_01.csv
4 pilot_tabletennis_01,tabletennis,singles,TODO(avidullu): match name,TODO(avidullu):
exact source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:20,10-12,,todo,deliver as pilot_tabletennis_01.csv
5
```

11. assistant (2026-06-08T08:39:11.490Z)

Now let me check for any references to the VLC rally-annotator and manual self-rating ingestion:

12. user (2026-06-08T08:39:12.619Z)

No files found

13. user (2026-06-08T08:39:13.149Z)

Found 2 files
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md

14. assistant (2026-06-08T08:39:15.813Z)

Let me search more broadly for any reference to different ingestion paths or partial labels:

15. user (2026-06-08T08:39:16.460Z)

Exit code 1
/usr/bin/bash: line 1: cd: C:\Users\avidu\Projectssports-data-collector: No such file or directory

16. user (2026-06-08T08:39:16.836Z)

Found 4 files
C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py
C:\Users\avidu\Projects\sports-data-collector\src\cli\ingest.py

17. assistant (2026-06-08T08:39:18.547Z)

Let me search for any documentation about self-rating or partial labels:

18. user (2026-06-08T08:39:19.148Z)

C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md:91:completeness. (A
future enhancement: a motion second-opinion over the raw video.)

19. assistant (2026-06-08T08:39:21.599Z)

Now let me check the `curate.py` to understand the import-local process:

20. user (2026-06-08T08:39:21.770Z)

```
1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")
30            self.db = SportsDatabase(db_path)
31            self.validator = DatasetValidator(config_path)
32
33        def _load_config(self) -> Dict[str, Any]:
34            if not os.path.exists(self.config_path):
35                print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36            return load_config(self.config_path)
37
38        def status(self):
39            """Displays status summary of all videos in the database."""
40            # Query total count by status via the public DB read API.
41            rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43            if not rows:
44                print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45                return
46
47            data = [[sport, status, cnt] for sport, status, cnt in rows]
48            print("\n=== Dataset Ingestion Status ===")
49            print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50            print()
51
52        def import_local(self, args):
53            """Manually registers a local video and annotation file into the database."""
54            sport_str = args.sport.lower()
55            if sport_str not in self.config.get("supported_sports", ["badminton"]):
56                print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
```

```

57         sys.exit(1)
58
59     # Validate video path
60     if not os.path.exists(args.video_path):
61         print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63     # Validate annotation file
64     if not os.path.exists(args.annotations_path):
65         print(f"Error: Annotations file '{args.annotations_path}' not found.")
66         sys.exit(1)
67
68     match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69     video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71     # Parse license
72     try:
73         license_type = LicenseType(args.license)
74     except ValueError:
75         print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76         license_type = LicenseType.UNKNOWN
77
78     is_commercial = self.validator.check_license_commercial(license_type)
79
80     # 1. Parse Annotations
81     annotations = []
82     try:
83         annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84     except Exception as e:
85         print(f"Error parsing annotations: {e}")
86         sys.exit(1)
87
88     # 2. Run Validation
89     is_valid = True
90     errors = []
91     if sport_str == "badminton":
92         is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93     elif sport_str == "tennis":
94         is_valid, errors = self.validator.validate_tennis_points(annotations)
95     elif sport_str == "cricket":
96         is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97     elif sport_str == "pickleball":
98         is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99     elif sport_str == "tabletennis":
100         is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102     if not is_valid:
103         print(f"Error: Annotations failed validation!")
104         for err in errors:
105             print(f" - {err}")
106         sys.exit(1)
107
108     # 3. Create Video Metadata
109     video_meta = VideoMetadata(
110         video_id=video_id,
111         sport=SportType(sport_str),
112         video_url=None,
113         local_path=args.video_path,
114         license_type=license_type,
115         license_url=None,
116         is_commercial=is_commercial,
117         status=CurationStatus.CURATED, # Custom local imports bypass staging by
default

```



```

118         match_name=match_name,
119         fps=args.fps,
120         resolution=args.resolution
121     )
122
123     # 4. Insert into Database
124     self.db.insert_video(video_meta)
125
126     count = 0
127     if sport_str == "badminton":
128         count = self.db.insert_badminton_rallies(annotations)
129     elif sport_str == "tennis":
130         count = self.db.insert_tennis_points(annotations)
131     elif sport_str == "cricket":
132         count = self.db.insert_cricket_deliveries(annotations)
133     elif sport_str == "pickleball":
134         count = self.db.insert_pickleball_rallies(annotations)
135     elif sport_str == "tabletennis":
136         count = self.db.insert_tabletennis_rallies(annotations)
137
138     print(f"\nSuccessfully imported local video '{match_name}':")
139     print(f"    - Video ID: {video_id}")
140     print(f"    - Sport: {sport_str}")
141     print(f"    - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142     print(f"    - Registered {count} annotated segments.\n")
143
144     # Per-video observability report (next to the video). Never raises.
145     from src.reporting import emit_import_report
146     src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147     report_paths = emit_import_report(
148         video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149         errors=[], count=count, sport=sport_str, source_format=src_fmt,
150         fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),

```

21. assistant (2026-06-08T08:39:24.225Z)

Let me get more of the curate.py file to understand the full import flow:

22. user (2026-06-08T08:39:24.768Z)

```

150         fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151     )
152     if report_paths:
153         print(f"    - Observability report: {report_paths.get('technical_md')}")
154         print(f"    - Customer summary      : {report_paths.get('customer_md')}\n")
155
156     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
157         """Parses simple JSON or CSV annotations files."""
158         parsed_items = []
159         ext = os.path.splitext(filepath)[1].lower()
160
161         if ext == '.json':
162             with open(filepath, 'r', encoding='utf-8') as f:
163                 data = json.load(f)
164
165             if not isinstance(data, list):
166                 raise ValueError("JSON file must contain a list of objects.")
167
168             # Uses the same safe casts as the CSV path: a missing/blank required
169             # time raises a clear ValueError (caught by the caller) instead of a
170             # bare KeyError, and optional ints degrade to sensible defaults.
171             for idx, obj in enumerate(data):

```

```

172         if sport == "badminton":
173             parsed_items.append(BadmintonRally(
174                 video_id=video_id,
175                 rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                 start_time=_safe_float(obj.get("start_time")),
177                 end_time=_safe_float(obj.get("end_time")),
178                 score_before=obj.get("score_before"),
179                 score_after=obj.get("score_after"),
180                 shots_count=_safe_int(obj.get("shots_count"), None),
181                 ending_reason=obj.get("ending_reason"),
182                 last_shot_outcome=obj.get("last_shot_outcome")
183             ))
184         elif sport == "tennis":
185             parsed_items.append(TennisPoint(
186                 video_id=video_id,
187                 point_number=_safe_int(obj.get("point_number"), idx + 1),
188                 start_time=_safe_float(obj.get("start_time")),
189                 end_time=_safe_float(obj.get("end_time")),
190                 set_score=obj.get("set_score"),
191                 game_score=obj.get("game_score"),
192                 server=obj.get("server"),
193                 ending_type=obj.get("ending_type")
194             ))
195         elif sport == "cricket":
196             parsed_items.append(CricketDelivery(
197                 video_id=video_id,
198                 over=_safe_int(obj.get("over"), 0),
199                 ball=_safe_int(obj.get("ball"), 0),
200                 start_time=_safe_float(obj.get("start_time")),
201                 end_time=_safe_float(obj.get("end_time")),
202                 batsman=obj.get("batsman"),
203                 bowler=obj.get("bowler"),
204                 runs=_safe_int(obj.get("runs"), None),
205                 wicket=bool(obj.get("wicket", False)),
206                 extras=_safe_int(obj.get("extras"), 0)
207             ))
208         elif sport == "pickleball":
209             parsed_items.append(PickleballRally(
210                 video_id=video_id,
211                 rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212                 start_time=_safe_float(obj.get("start_time")),
213                 end_time=_safe_float(obj.get("end_time")),
214                 score_before=obj.get("score_before"),
215                 score_after=obj.get("score_after"),
216                 shots_count=_safe_int(obj.get("shots_count"), None),
217                 ending_reason=obj.get("ending_reason"),
218                 last_shot_outcome=obj.get("last_shot_outcome")
219             ))
220         elif sport == "tabletennis":
221             parsed_items.append(TableTennisRally(
222                 video_id=video_id,
223                 rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224                 start_time=_safe_float(obj.get("start_time")),
225                 end_time=_safe_float(obj.get("end_time")),
226                 score_before=obj.get("score_before"),
227                 score_after=obj.get("score_after"),
228                 shots_count=_safe_int(obj.get("shots_count"), None),
229                 ending_reason=obj.get("ending_reason"),
230                 last_shot_outcome=obj.get("last_shot_outcome")
231             ))
232     elif ext == '.csv':
233         with open(filepath, 'r', encoding='utf-8') as f:
234             reader = csv.DictReader(f)
235             # Normalize headers
236             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if

```

```

reader.fieldnames else []
237
238     for idx, row in enumerate(reader):
239         if sport == "badminton":
240             parsed_items.append(BadmintonRally(
241                 video_id=video_id,
242                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                 start_time=_safe_float(row.get("start_time")),
244                 end_time=_safe_float(row.get("end_time")),
245                 score_before=row.get("score_before"),
246                 score_after=row.get("score_after"),
247                 shots_count=_safe_int(row.get("shots_count"), None),
248                 ending_reason=row.get("ending_reason"),
249                 last_shot_outcome=row.get("last_shot_outcome")
250             ))
251         elif sport == "tennis":
252             parsed_items.append(TennisPoint(
253                 video_id=video_id,
254                 point_number=_safe_int(row.get("point_number"), idx + 1),
255                 start_time=_safe_float(row.get("start_time")),
256                 end_time=_safe_float(row.get("end_time")),
257                 set_score=row.get("set_score"),
258                 game_score=row.get("game_score"),
259                 server=row.get("server"),
260                 ending_type=row.get("ending_type")
261             ))
262         elif sport == "cricket":
263             parsed_items.append(CricketDelivery(
264                 video_id=video_id,
265                 over=_safe_int(row.get("over"), 0),
266                 ball=_safe_int(row.get("ball"), 0),
267                 start_time=_safe_float(row.get("start_time")),
268                 end_time=_safe_float(row.get("end_time")),
269                 batsman=row.get("batsman"),
270                 bowler=row.get("bowler"),
271                 runs=_safe_int(row.get("runs"), None),
272                 wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273                 extras=_safe_int(row.get("extras"), 0)
274             ))
275         elif sport == "pickleball":
276             parsed_items.append(PickleballRally(
277                 video_id=video_id,
278                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
279                 start_time=_safe_float(row.get("start_time")),
280                 end_time=_safe_float(row.get("end_time")),
281                 score_before=row.get("score_before"),
282                 score_after=row.get("score_after"),
283                 shots_count=_safe_int(row.get("shots_count"), None),
284                 ending_reason=row.get("ending_reason"),
285                 last_shot_outcome=row.get("last_shot_outcome")
286             ))
287         elif sport == "tabletennis":
288             parsed_items.append(TableTennisRally(
289                 video_id=video_id,
290                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
291                 start_time=_safe_float(row.get("start_time")),
292                 end_time=_safe_float(row.get("end_time")),
293                 score_before=row.get("score_before"),
294                 score_after=row.get("score_after"),
295                 shots_count=_safe_int(row.get("shots_count"), None),
296                 ending_reason=row.get("ending_reason"),
297                 last_shot_outcome=row.get("last_shot_outcome")
298             ))
299     else:

```

```

300         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
301
302     return parsed_items
303
304 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
305     """Download one video at best resolution, retrying transient failures.
306
307     A below-min-height result almost always means YouTube rate-limited the
308     request and served a low fallback (not that the video lacks HD), so we
309     retry with a growing backoff. Returns (DownloadResult, None) on success,
310     or (None, failure_reason) once retries are exhausted. A below-min file is
311     discarded each attempt so it never lands in the DB.
312     """
313     attempts = max(1, args.retries + 1)
314     reason = "unknown error"
315     for i in range(attempts):
316         result = download_best_video(
317             url, dest_path,
318             max_height=max_height, max_size_mb=max_size_mb,
319             cookies_from_browser=args.cookies_from_browser,
320             cookies_file=args.cookies,
321             player_client=args.player_client,
322         )
323         if result.success and (result.height is None or result.height >=
args.min_height):
324             return result, None
325
326         if not result.success:
327             reason = result.error or "unknown error"
328         else:
329             reason = (f"got {result.resolution_label} (< min-height
{args.min_height})p) ? "
330                     f"YouTube served a low format (rate-limited?)")
331             try:
332                 os.remove(result.path)
333             except OSError:
334                 pass
335
336             if i < attempts - 1:
337                 backoff = args.retry_backoff * (i + 1)
338                 print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
339                 time.sleep(backoff)
340     return None, reason
341
342 def fetch(self, args):
343     """(Re)download videos at best available resolution, updating the DB in place.
344
345     This is the in-place upgrade path: rows, rally annotations, and curation
346     status are all preserved ? only the video file plus its `local_path`,
347     `resolution`, and `fps` are refreshed. By default it skips videos that
348     already have a local file at or above `--min-height`, so re-runs are cheap
349     and resumable; `--force` re-fetches everything.
350     """
351     storage = self.config.get("video_storage", {})
352     local_dir = storage.get("local_dir", "./data/videos")
353     max_download_size_mb = storage.get("max_download_size_mb")
354     max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
355     os.makedirs(local_dir, exist_ok=True)
356
357     commercial_only = not args.include_noncommercial
358     if args.status == "all":
359         statuses = [CurationStatus.CURATED, CurationStatus.STAGING]

```

```

360         else:
361             statuses = [CurationStatus(args.status)]
362
363             supported = self.config.get("supported_sports", ["badminton"])
364             sport_strs = [args.sport] if args.sport else supported
365
366             # Collect target videos.
367             videos = []
368             for sport_str in sport_strs:
369                 try:
370                     sport = SportType(sport_str.lower())
371                 except ValueError:
372                     print(f"Warning: skipping unknown sport '{sport_str}'.")
373                     continue
374                 for status in statuses:
375                     videos.extend(self.db.get_videos_by_status(status, sport))
376             if args.video_id:
377                 videos = [v for v in videos if v.video_id == args.video_id]
378
379             if not videos:
380                 print("\nNo matching videos to fetch.\n")
381                 return
382
383             results = [] # (video_id, action, detail)
384             for video in videos:
385                 if commercial_only and not video.is_commercial:
386                     results.append((video.video_id, "skip", "non-commercial"))
387                     continue
388                 if not video.video_url or "watch?v=example" in video.video_url:
389                     # Either no URL, or the parser's placeholder (the ShuttleSet catalog
390                     # had no URL for this match) ? nothing to download.
391                     results.append((video.video_id, "skip", "no real source URL"))
392                     continue
393
394                 dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
395
396                 # Skip if we already have a good-enough local file (unless --force).
397                 if not args.force and os.path.exists(dest_path):
398                     _, height, _ = probe_resolution_fps(dest_path)
399                     if height is not None and height >= args.min_height:

```

23. assistant (2026-06-08T08:39:27.278Z)

Let me look for any information about the badminton-highlight-indexer repo and how it interacts with the collector:

24. user (2026-06-08T08:39:27.616Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer'

25. user (2026-06-08T08:39:29.666Z)

```

docs\EVALUATION.md
docs\TRACKNET_WSL_SETUP.md
docs\GOLDEN_SET_IMPLEMENTATION.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\GOLDEN_SET_VENDURING.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
docs\archives\README.md
docs\BACKLOG.md

```

docs\CODE_MAP.md
docs\DEFERRED_HARDENING_PLAN.md
docs\PMF_MARKET_RESEARCH.md
docs\ROADMAP.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\decisions\DECISIONS.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\past_projects\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\quality-iterations\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\COMMERCIALIZATION.md
docs\PLATFORM_ARCHITECTURE.md
docs\README.md

26. assistant (2026-06-08T08:39:31.410Z)

Let me read the GOLDEN_SET_IMPLEMENTATION and other key indexer docs:

27. user (2026-06-08T08:39:31.891Z)

```
1      # Golden-Set Subproject ? Design & Implementation Overview
2
3      **Status:** In progress . **Date:** 2026-05-30
4      **Companion docs:** design rationale + vendor shortlist in
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (ADR-006); scoring mechanics in
[EVALUATION.md](EVALUATION.md).
5
6      This is the **implementation tracker** for turning the golden-set/vendor *design* into
shipped code. It's split into small, independently-reviewable PRs. The whole subproject is
**vendor-agnostic by construction** ? only one slice (GS-3, `HumanVendorProvider`) depends on
which vendor you pick offline; everything else can land first.
7
8      > **Update (2026-06-02):** Vendor selected for the **initial pilot ? Roora**
(Bengaluru). The **annotation guideline, intake-manifest template, and NDA** are authored in the
collector repo `docs/annotation/`, and the collector's `import-local` ingestion now covers the
**3 pilot sports** (badminton, pickleball, table tennis). On the indexer side, GS-1
(`file_provider.py`), GS-2 (`backend/eval/regression.py` + `run_regression.py`), and the GS-4
oracle scaffold (`automated_provider.py`) have landed; the remaining slice is the vendor-
dependent **GS-3 `HumanVendorProvider`**, parked until the Roora pilot confirms quality.
9
10     ---
11
12     ## Architecture in one picture
13
14     ```
15         video_ref ??? AnnotationProvider.submit()/poll()/fetch() ???
List[(start,end)]   ground truth
16                 ?           ?           ?           ?
17         FileProvider  HumanVendor  Automated(oracle)  ?
18         (GS-1)        (GS-3)        (GS-4)
backend.eval.metrics.score() ??? existing, unchanged
19
20         production pipeline ??? DB ??? backend.eval.harness.get_predictions() ???
List[(start,end)]   predictions
21
22                                     regress(video, tier) (GS-2)
compares the two
23     ```
24
25     **The keystone:** a provider's `fetch()` returns the **same `List[(start, end)]`** that
```

```

`eval.annotations.load_annotations()` already returns and `eval.metrics.score()` already
consumes. So one scorer grades all three of: product-vs-GT, vendor-vs-answer-key, and annotator-
vs-annotator (IAA). No new scoring code ? only new *sources* of intervals.
26
27     ---
28
29     ## PR breakdown
30
31     | Slice | Branch | Scope | Depends on | Vendor-dependent? |
32     |---|---|---|---|---|
33     | **GS-1** | `feat/annotation-provider-registry` | `AnnotationProvider` ABC + registry +
`FileProvider` + tests | ? | No |
34     | **GS-2** | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
35     | **GS-3** | `feat/human-vendor-provider` | `HumanVendorProvider` (submit/poll/fetch
against the chosen vendor's API or file-drop), DPA/secure-link handling | GS-1, **your vendor
pick** | **Yes** |
36     | **GS-4** | `feat/automated-oracle-provider` | `AutomatedProvider` fast-tier oracle ?
**interface stub** (injected labeler + fake; concrete model = backlog) | GS-1, GS-2 | No |
37     | **GS-5** | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI,
scheduled run | GS-2, GS-4 | No |
38
39     Each row is one PR. GS-1 ? GS-2 ? GS-4 ? GS-5 can proceed **now**, independent of vendor
selection. GS-3 is parked until you've chosen a vendor (then it's a thin adapter behind the GS-1
interface).
40
41     ---
42
43     ## Slice details
44
45     ### GS-1 ? Provider registry + FileProvider ? (this PR)
46     - `backend/annotations/base.py` ? `AnnotationProvider` ABC (`submit`/`poll`/`fetch` +
`annotate()` convenience) and `annotation_registry`, mirroring `backend/providers` and
`backend/indexer` registry patterns.
47     - `backend/annotations/file_provider.py` ? `FileProvider`: the "job" is a CSV that
already exists (formalizes today's manual/ShuttleSet flow); reuses
`eval.annotations.load_annotations`.
48     - Tests assert the registry contract **and** the keystone (provider output is directly
scorable by `metrics.score()`).
49     - **Design choice:** `fetch()`/`annotate()` raise loudly on a missing/incomplete job
rather than returning `[]`, so a broken label source can't silently masquerade as "0 rallies"
and skew a regression to green.
50
51     ### GS-2 ? Regression runner (next)
52     `regress(video, tier="fast", tolerance=0.05)`: get GT from
`annotation_registry.get(tier)`, read predictions via the existing `eval.harness`, score with
the existing `metrics`, compare against a per-video `baseline.json`, and flag a regression when
precision/recall drop beyond tolerance. Pure orchestration over existing parts + a small
baseline store. CLI: `python run_regression.py --video ... --tier file`.
53
54     ### GS-3 ? HumanVendorProvider (after your vendor pick)
55     A concrete provider that submits a (licensed/owned-only) video to the chosen vendor and
fetches the returned rally CSV. Carries the privacy controls from the design doc (secure time-
limited links, delete-on-delivery, DPA). Thin adapter ? the interface already exists from GS-1.
56
57     ### GS-4 ? AutomatedProvider (fast-tier oracle) ?? oracle rule
58     An independent strong labeler for the fast/CI tier. **Must be a different model lineage
than whatever production uses** (else shared blind spots ? false green). A fast-tier failure is
a *smoke alarm* that escalates to the GS-3 human tier, never an authoritative verdict on its
own.
59
60     ### GS-5 ? Regression triggers
61     Wire `regress()` into CI (pre-merge gate on pipeline changes), an on-demand CLI (the
"surprise regression"), and an optional scheduled run. Includes a deliberate re-snapshot path so
legitimate production improvements can rebaseline instead of raising false regressions.

```

```

62
63     ---
64
65     ## Conventions
66     - Branch per slice, one PR each, tests green before PR, full suite must stay green.
67     - New runtime deps: none through GS-2 (GS-3 may add a vendor SDK; GS-4 may reuse the
existing `google-genai` for a Gemini oracle).
68     - All ground-truth video stays licensed/owned ? see the privacy section of the
design doc.
69

```

28. user (2026-06-08T08:39:32.137Z)

```

1      # Architecture Decision Records
2
3      ## ADR-001: Precise shuttle tracking via self-hosted TrackNetV4/WASB on WSL2
4
5      Date: 2026-05-25
6      Status: Accepted
7
8      ### Context
9
10     The pipeline uses an AI multimodal provider (Gemini) for semantic rally segmentation.
11     We need precise frame-level detection of the moving shuttlecock (and, generically,
12     balls for other sports) to produce higher-quality candidate rallies and a fine-tuning
13     signal.
14
15     We evaluated whether a commercially available model/API could do this. Findings:
16
17     - Multimodal LLM APIs (Gemini, Claude, GPT-class) are the wrong tool for precise
18       localization. They reason well about what is happening (rally vs. dead time) but
19       cannot reliably track a tiny, fast (~up to 400 km/h), motion-blurred shuttle frame by
20       frame.
21     - The precision leaders are purpose-built trajectory models, which are open source
and
22       self-hosted, not big-vendor APIs:
23     - TrackNetV4 (TensorFlow) ? badminton-specific, heatmap + motion-attention,
24       highest reported shuttle accuracy; ships `src/predict.py` producing per-frame (X, Y)
25       coordinate CSV. https://github.com/AR4152/TrackNetV4
26     - WASB (PyTorch) ? multi-sport strong baseline (badminton/tennis/volleyball/?),
27       fits our `SportProfile` abstraction; needs a custom-video inference wrapper.
28       https://github.com/nttcom/WASB-SBDT
29     - True turnkey commercial APIs (Roboflow hosted models) exist but cap below a tuned
30       TrackNet; enterprise systems (Hawk-Eye, SkillCorner, TRACAB) are not API-accessible.
31
32     ### Decision
33
34     1. Adopt self-hosted TrackNetV4 as the primary precise shuttle detector, with
WASB
35       as the multi-sport fallback.
36     2. Run these models under WSL2 (Ubuntu), not native Ubuntu. The entire indexer
37       already runs on Windows; WSL2 keeps a single dev environment and quarantines the
38       TensorFlow/PyTorch dependencies. Native Ubuntu would require dual-boot and migrating
39       the whole project for marginal gain.
40     3. Integrate via a subprocess adapter: a detector module that shells out to the WSL
41       `predict.py`, reads the coordinate CSV, and converts trajectory ? candidate rallies
42       feeding the existing stitching flow. This keeps the TF dependency fully isolated from
43       the main Python environment.
44
45     ### Consequences
46
47     - WSL I/O caveat: `/mnt/c` access is slow. Stage match videos inside the WSL
48       filesystem (e.g. `~/clips/`) before inference rather than reading large mp4s across
the
49       Windows?WSL boundary.

```



```

50 - TrackNetV4 (TensorFlow) and WASB (PyTorch) live in separate conda envs.
51 - Next step after integration: manual video annotation ? fine-tune the detector.
52
53 ---
54
55 ADR-002: Use the sports-data-collector's rally annotations as a golden eval +
temporal-segmenter training set
56
57 Date: 2026-05-26
58 Status: Accepted
59
60 Context
61
62 A sibling project, sports-data-collector, curates sports videos and stores per-rally
63 [start, end) timestamps (plus stroke counts, scores) in an SQLite DB, with a
64 commercial-license flag per source. We asked whether this data could improve the
65 indexer's rally detection, and for which task.
66
67 Key findings:
68
69 - The annotations are temporal-only (when rallies start/end). They do not
contain
70 per-frame shuttle (x, y) coordinates, so they cannot train the shuttle detector
71 (TrackNetV4/WASB ? see ADR-001), which needs coordinate labels.
72 - They are an excellent golden evaluation set and a training signal for the
73 temporal rally segmenter (active vs. dead play over time) ? the stage that is
today
74 a hand-tuned heuristic, not a learned model.
75 - The collector's five sports tables (badminton_rallies, tennis_points,
76 cricket_deliveries, pickleball_rallies, tabletennis_rallies) all share a
77 (video_id, ordinal, start_time, end_time) shape, and the indexer already has
78 sport_registry / segmenter_registry. So a generic (start, end) bridge serves
79 every sport.
80 - The two repos key videos differently: the collector by a human video_id, the indexer
81 by file MD5. A re-encode/re-download changes the bytes, hence the MD5.
82
83 Decision
84
85 1. Treat the collector's rally timestamps as (a) the golden eval set for rally
86 segmentation and (b) the labels for training a temporal segmenter ? explicitly
87 not as shuttle-detector training data.
88 2. Bridge the repos with a sport-agnostic export (curate export in the collector):
89 per-video start,end CSVs (the indexer's eval format) + a manifest.json that pairs
90 each CSV with its video file. Gate on is_commercial so only commercially usable
data
91 flows through.
92 3. Honour the MD5 keying contract: evaluation/processing must run on the exact video
93 file recorded in the manifest; acquisition (ADR-003) therefore precedes processing.
94
95 Consequences
96
97 - The collector becomes the producer of labels; the indexer the consumer (processing
+
98 scoring + training). Neither takes a hard dependency on the other beyond the manifest.
99 - Scoring is free and deterministic (reads the DB; no API). See docs/EVALUATION.md.
100 - Only invest in frame-level coordinate labels (for the detector) if evaluation
shows
101 detection, not segmentation, is the bottleneck.
102
103 ---
104
105 ADR-003: Video acquisition ? always best resolution, upgrade in place, authenticated
cookies
106
107 Date: 2026-05-26

```

```

108     **Status:** Accepted
109
110     ### Context
111
112     The collector's initial crawl downloaded videos with `yt-dlp -f worstvideo` (256×144),
113     mislabelled as 1080p. The indexer needs 720p MP4 for meaningful YOLO/shuttle detection.
114     Re-acquiring full-resolution sources surfaced several hard-won realities:
115
116     - The ShuttleSet corpus **tops out at 1080p** (no 4K); "best available" resolves to
1080p.
117     - Authenticating with a **YouTube Premium** account is required for reliable,
unthrottled
118     downloads ? anonymous downloads are throttled (~1.3 MiB/s) and **non-deterministic**
119     (identical commands variously yielded 1080p DASH, 1080p HLS, or a 144p fallback).
120     - **Chrome** cookies can't be read on Windows (app-bound encryption / DPAPI, yt-dlp
121     #10927); **Firefox** cookies read cleanly even while open.
122     - The `web`/`web_safari` player clients are forced into **SABR streaming**, which strips
123     download URLs and silently drops to 144p (yt-dlp #12482). yt-dlp's `default` client
124     serves real downloadable 1080p DASH with auth cookies.
125     - Bulk bursts trip YouTube **rate-limiting** (a 22-video burst failed all at once,
though
126     each worked in isolation).
127
128     ### Decision
129
130     1. **Always fetch the best resolution available** (`bv*+ba/b -S res,...`, merged to
MP4);
131     ffprobe the result and store the *actual* width/height/fps ? never assume.
132     2. **Upgrade in place, never clean-slate:** the `fetch` command re-downloads a video and
133     updates only `local_path`/`resolution`/`fps`, preserving every rally annotation and
134     curation status (deleting would cascade-delete the valuable labels).
135     3. Use **authenticated cookies** (`--cookies-from-browser firefox` / `--cookies file`)
and
136     the **`default` player client** ; enable the EJS challenge solver (deno).
137     4. **Throttle bulk runs** (`--sleep` + per-video `--retries`/`backoff`) and treat a
138     below-min-height download as a **failure** (discard, don't commit) so a flaky low-res
139     result never overwrites a good entry.
140
141     ### Consequences
142
143     - All 22 fetchable ShuttleSet matches are now true 1080p (one match, shuttleset_12, has
no
144     source URL in the catalog and remains video-less but keeps its annotations).
145     - Acquisition is reproducible and resumable; the ~26 GB corpus is kept outside git.
146     - Future non-YouTube or 4K sources are supported by the same code (optional
`max_height`).
147
148     ---
149
150     ## ADR-004: Add an offline, API-free *learned* rally segmenter as a first-class mode
151
152     **Date:** 2026-05-26
153     **Status:** Proposed (substrate pending data)
154
155     ### Context
156
157     The default high-quality path (`yolo_hybrid`) refines YOLO candidate windows with
**Gemini**,
158     which costs money and requires network access. With a now-1080p, commercially-licensed,
159     rally-annotated corpus (ADR-002/003), we can learn the refinement step offline instead.
160
161     Crucially, the **API-free signal already exists and is already persisted**:
`yolo_hybrid`
162     Stage 1 is pure YOLO (player tracking ? per-window motion features: peak/mean velocity,
163     active-frame count, track-id count) stored in `candidate_segments.stats`. Gemini is only

```

```

164     the Stage-2 refiner. So the rally annotations can **train a classifier to replace
Gemini**,
165     with no external API and **no new labeling**.
166
167     Three candidate substrates were weighed:
168
169     - **Tune `motion` thresholds** on the annotations ? trivial, fully offline, but a low
170       ceiling (pixel motion can't separate rally from crowd/pan/replay). Use as a *floor*.
171     - **YOLO features ? trained classifier** ? reuses already-stored, semantically
meaningful
172       player-motion features; medium-high ceiling; moderate effort. *Recommended primary.*
173     - **WASB shuttle-motion ? rally detector** ? highest ceiling (the shuttle defines play),
174       fully offline but heaviest (needs the WSL2 WASB runner, ADR-001); can't train WASB
175       itself (no xy labels), only a rally layer on its output.
176
177     ### Decision
178
179     1. Build an **offline, API-free learned segmenter** as a new registry mode (e.g.
180       `yolo_learned`): label detector windows by overlap with GT rallies, train a
lightweight
181       classifier on the persisted features, emit final segments with **zero API calls**,
and
182       score it with the existing harness against `motion` and a Gemini reference.
183     2. **Choose the substrate from data, not assumption.** A learned refiner can only
sharpen
184       the windows the detector found, so the deciding metric is the detector's **candidate-
stage
185       recall** vs. the annotations: high recall ? build on YOLO features; low recall ?
bring in
186       WASB's shuttle signal to raise the ceiling.
187     3. Keep the Gemini-backed `yolo_hybrid` as an **upper-bound reference**, not a
production
188       dependency.
189
190     ### Consequences
191
192     - The final tool gains a **self-contained, offline, free** rally mode trained on its own
193       curated data ? runnable air-gapped, improvable as more matches are annotated.
194     - The approach is **sport-generic**: the same label?feature?classifier machinery extends
to
195       tennis/cricket once those have annotations.
196     - The full paid 22-video Gemini evaluation is **not** run; spend is limited to a single
197       reference video.
198
199     ### Update (2026-05-26): Phase 3 evidence ? substrate is WASB, not YOLO
200

```

29. assistant (2026-06-08T08:39:35.088Z)

Now let me check if there's any mention of the rally-annotator tool in the collector repo or elsewhere:

30. user (2026-06-08T08:39:35.877Z)

C:/Users/avidu/Projects/rally-annotator

31. assistant (2026-06-08T08:39:37.948Z)

Good! Let me check the rally-annotator repo:

32. user (2026-06-08T08:39:38.298Z)

C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md
C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md

C:/Users/avidu/Projects/rally-annotator\README.md
C:/Users/avidu/Projects/rally-annotator\CHANGELOG.md

33. user (2026-06-08T08:39:38.562Z)

```
total 33
drwxr-xr-x 1 avidu 197609    0 Jun  8 13:56 .
drwxr-xr-x 1 avidu 197609    0 Jun  7 12:37 ..
drwxr-xr-x 1 avidu 197609    0 Jun  8 13:57 .git
-rw-r--r-- 1 avidu 197609  123 Jun  7 06:58 .gitignore
-rw-r--r-- 1 avidu 197609  2635 Jun  8 13:56 CHANGELOG.md
-rw-r--r-- 1 avidu 197609  1066 Jun  7 06:57 LICENSE
-rw-r--r-- 1 avidu 197609  6847 Jun  8 13:56 README.md
drwxr-xr-x 1 avidu 197609    0 Jun  8 13:45 docs
drwxr-xr-x 1 avidu 197609    0 Jun  8 13:56 vlc
```

34. user (2026-06-08T08:39:41.249Z)

```
1      # Rally Annotator
2
3      A tiny **VLC plugin** to hand-label rally **start/end + point-stop reason** while you
watch a match,
4      for **net-separated racquet sports** ? badminton, tennis, table tennis, pickleball,
padel.
5
6      You watch the video in VLC, pause/scrub freely, and click **Mark START** / **Mark END**
(with a
7      reason). It writes one CSV row per rally ? clean ground-truth labels for
training/evaluating rally
8      (point) segmentation models, or just for cutting highlights.
9
10     ``
11     rally_number,start_time,end_time,ending_reason,sport
12     1,8.800,11.500,winner,badminton
13     2,24.389,46.589,unforced_error,badminton
14     ``
15
16     Times are **decimal seconds**. See [docs/CSV_FORMAT.md](docs/CSV_FORMAT.md).
17
18     ## Why
19     Labeling rally boundaries is the slow, expensive prerequisite for any rally-detection
model. Most tools
20     make you transcribe timestamps by hand. This lets a rater do it **inside the player they
already use**,
21     pausing to the exact frame before each mark ? turning "watch a match" into "produce
golden labels."
22
23     ## Install
24     1. Copy `vlc/rally_annotator.lua` into your VLC Lua **extensions** folder:
25         - **Windows:** `%APPDATA%\vlc\lua\extensions\`
26         - **macOS:** `~/Library/Application Support/org.videolan.vlc/lua/extensions/`
27         - **Linux:** `~/.local/share/vlc/lua/extensions/`
28         (create the `extensions` folder if it doesn't exist)
29     2. In VLC: **Tools ? Plugins and extensions ? Reload extensions** (or just restart VLC).
30     3. Enable it from the **View** menu ? **Rally Annotator**. A small dialog opens and
stays open while you watch.
31
32     Requires **VLC 3.0.x**. (VLC 4.0 changed the Lua input/listener API; targeting 3.x for
now.)
33
34     ## Use
35     1. Pick the **Sport** (it stays set across rallies).
36     2. When a rally begins, click **Mark START** ? it snapshots the current playback time
into the **Start** field
37         (pause/scrub first for frame accuracy; you can also edit the field by hand).
```

```

38     3. When the rally ends, click **Mark END** (fills the **End** field). This *arms* the
rally; nothing is written yet.
39     4. Choose the **Ending reason** ? it is **required** ? and click **Save Rally** to write
one CSV row.
40     The reason **resets after every save**, so it can't silently reuse the previous
rally's reason.
41     5. **Recent rallies** list: select a row, then **Edit selected** (loads it back into the
fields to fix
42         start/end/reason/sport ? **Save changes**) or **Delete selected**. **Undo last**
removes the most recent row ?
43         the button shows which one, e.g. `Undo last (#7)`, and the status panel shows that
row's details ? or clears an
44         in-progress mark, or cancels an edit.
45     6. **Help** button ? usage + an [ending-reason decision guide](docs/ENDING_REASONS.md),
rendered inside the dialog.
46
47     **Output:** `<video-stem>.rallies.csv` next to the video (falls back to your home dir if
the path can't be
48     resolved). Re-opening the extension **continues** rally numbering from the existing file
? no duplicate IDs.
49
50     **Resuming a half-finished video:** labels are saved to that CSV as you go. Open the
**same video** and enable the
51     extension and it **reloads your existing rallies** (shown in the Recent list) and
continues numbering ? so you can
52     stop and pick up later. Easiest flow: **open the video first, then enable the
extension.** If you switch videos with
53     the dialog already open, click **Refresh** to point at the current video and load its
rallies. (Playback *position*
54     isn't restored ? scrub to where you stopped.)
55
56     > **Where's the help / "About" text?** The empty **About ?lua** box under **Tools ?
Plugins and extensions ?
57     > Add-ons** is VLC's own add-on info dialog; its body ("Lua script") is a constant baked
into VLC and **can't be
58     > set by an extension**. Use the **Help** button inside the Rally Annotator dialog
instead. (The descriptor's
59     > description does show in the *Active Extensions* tab's **"More information"** dialog.)
60
61     ## Sports & taxonomy
62     Net-separated racquet sports share a forced/unforced-error point-stop taxonomy, so one
tool covers them all:
63
64     | Sport | Notes |
65     |---|---|
66     | badminton | the reference sport |
67     | tennis | service_fault = fault/double-fault; let supported |
68     | table_tennis | let (net serve) supported; fast cadence |
69     | pickleball | service_fault for faults; treat "let" per local rules |
70     | padel | net-separated; rally boundaries as in tennis |
71
72     `ending_reason` ? `{winner, forced_error, unforced_error, service_fault, let, other}` ?
a shared vocabulary
73     across these sports. Keep to these values for clean downstream aggregation.
74
75     ### What the ending reasons mean
76     Every reason **except `winner`** is charged to the side that **lost** the rally.
77
78     | Reason | When the rally ended because? |
79     |---|---|
80     | `winner` | the last shot landed **in** and went unreturned (opponent couldn't reach it
/ only waved). A clean ace counts here. |
81     | `forced_error` | the loser **missed** (out or into the net) while **under pressure** ?
stretched, rushed, jammed, handling the opponent's pace/spin/depth. |
82     | `unforced_error` | the loser **missed a routine shot** they had time **and** position

```

```

to make, with little or no pressure. |
83     | `service_fault` | the point ended **on the serve** ? into the net, out of the service
box, illegal action/foot fault, or a double fault. |
84     | `let` | the rally is **replayed** under the rules, no point scored (e.g. a
tennis/table-tennis serve net-cord that's otherwise good, outside interference). |
85     | `other` | none of the above ? occluded footage, injury/retirement, penalty, hindrance.
Use sparingly. |
86
87     **The cases people ask about:** a ball/shuttle landing **out** is the *hitter's* error ?
`forced_error` only if they
88     were under pressure, otherwise `unforced_error` (it's **never** automatically forced,
and never a winner for the
89     opponent). **Into the net** during a rally is the same; a **serve** into the net is
`service_fault`. When unsure
90     forced-vs-unforced, **default to `unforced_error`**. See the full decision guide with
examples and per-sport rules
91     in **[docs/ENDING_REASONS.md](docs/ENDING_REASONS.md)** (also available via the in-
plugin **Help** button).
92
93     ## Roadmap
94     - [ ] Live-test the v1.3 dialog in VLC (two-step Save, required/non-sticky reason,
editable times, Recent-rallies
95         Edit/Delete) across all five sports.
96     - [ ] Optional per-sport reason presets / hotkeys.
97     - [ ] Alternative front-ends for power users / remote raters: a `python-vlc` + Tk/Qt app
with global keyboard
98         shortcuts (S/E/I?6/U), and a zero-install HTML5 `` page that exports the
same CSV.
99     - [ ] Optional extra columns (e.g. `shots_count`, server/receiver) behind a toggle.
100
101     ## Contributing
102     Issues and PRs welcome ? especially live test reports per sport/OS and small UX fixes.
The plugin is a single
103     self-contained Lua file (`vlc/rally_annotator.lua`) using only documented VLC 3.x Lua
APIs.
104
105     ## License
106     MIT ? see [LICENSE](LICENSE).
107

```

35. user (2026-06-08T08:39:41.347Z)

```

1     # Output CSV format
2
3     One row per rally. Header is written once when the file is created.
4
5     ...
6     rally_number,start_time,end_time,ending_reason,sport
7     1,8.800,11.500,winner,badminton
8     2,24.389,46.589,unforced_error,badminton
9     3,49.183,54.683,let,badminton
10    ...
11
12    | Column | Type | Notes |
13    |---|---|---|
14    | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15    | `start_time` | float | rally start, **decimal seconds** from the video start |
16    | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17    | `ending_reason` | enum | `winner` / `forced_error` / `unforced_error` /
`service_fault` / `let` / `other` ? see the decision guide in
[ENDING_REASONS.md](ENDING_REASONS.md) |
18    | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
19
20    Notes:
21    - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.

```

```

22     - Rows are appended live; **Undo last** truncates the last row (one level).
23     - Consumers should read by **column name** (the header is stable but column *order* may
evolve; extra
24     columns may be added later). Most CSV `DictReader`-style loaders already tolerate
extra columns.
25
26     ## Ingesting the labels
27
28     This CSV is intentionally compatible with common rally-segmentation tooling that reads
29     `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
30
31     ```bash
32     # Generic: any tool that takes a labels CSV of start/end seconds.
33
34     # Example ? score a detector against these labels, or slice clips by them
35     # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
36     python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
--fps 59.94 --frame-width 1920
37     python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
--rallies all
38
39     # Example ? register as curated golden data in sports-data-collector:
40     python -m src.cli.curate import-local --sport badminton --video-path match.mp4
--annotations-path match.rallies.csv
41     ```
42
43     The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
44

```

36. assistant (2026-06-08T08:39:44.171Z)

Perfect! Now let me check one more thing - see how the rally-annotator's CSV differs from the canonical format expected by the collector:

37. user (2026-06-08T08:39:44.325Z)

```

1     # Ending reasons ? what they mean & how to label them
2
3     The `ending_reason` column records **why the rally (point) ended**, judged from the
4     **last shot**. There are six values, shared across all the net-separated racquet sports
5     this tool supports (badminton, tennis, table tennis, pickleball, padel):
6
7     ```
8     winner | forced_error | unforced_error | service_fault | let | other
9     ```
10
11     **One core principle:** every reason **except `winner`** is charged to the player/side
12     that **LOST** the rally. `winner` is the only one that credits the side that **won** it.
13
14     This guide is written for a human rater clicking through a match. Where a rule comes
from
15     the laws of the sport it is marked **[rule]**; where it is an analytics *convention* (a
16     judgement call that charters agree to apply consistently) it is marked **[convention]**.
17
18     ---
19
20     ## The decision procedure (top to bottom ? stop at the first match)
21
22     1. **Was the rally replayed under the rules, with no point scored?**
23         (e.g. a tennis/padel serve that clips the net and lands in the correct box; a table-
tennis
24         serve that touches the net and is otherwise good; outside interference; receiver not
ready.)
25         ? **`let`**. *A net-cord during open rally play that lands in is NOT a let ? keep
going.*

```

```

26
27     2. **Did the point end on the SERVE because of a server fault?**
28     (serve into the net, serve outside the correct service court, illegal service action
/ foot
29     fault, or a tennis/padel **double fault**.)
30     ? **`service_fault`**.
31
32     3. **Did the winning side's last shot land IN and go unreturned** ? the opponent
couldn't
33     reach it, or only waved at it without meaningful contact? (A clean untouched serve /
**ace**
34     counts here.) ? **`winner`**.
35
36     4. **Otherwise the rally ended on a MISS by the loser** (ball/shuttle **out**, **into
the net**,
37     or not legally returned). Now judge **pressure**:
38
39     > **The pressure test:** Did the loser have **both enough time AND court position**
to make a
40     > routine return, given the opponent's *immediately preceding* shot (its placement,
pace,
41     > power, spin, depth, angle)? Ask: "Would a typical competent player at this level
be
42     > expected to make that shot?" **[convention]**
43
44     - **Yes** ? had time and position, missed a routine ball ? **`unforced_error`**.
45     - **No** ? time or position was taken away; the miss was induced ?
**`forced_error`**.
46
47     5. **Can't judge it, or it ended off-court** (occluded footage, injury/retirement,
code/point
48     penalty, hindrance, equipment failure) ? **`other`**. Use sparingly.
49
50     **Tie-breaker [convention]:** when you genuinely can't tell forced from unforced,
**default to
51     **`unforced_error`**. Only mark **`forced_error`** when you can *point to the specific
pressure* (the
52     wide/fast/deep/spinny ball that removed the player's time or position). This keeps
**`forced_error`**
53     a positively-evidenced label and stops it from inflating. The forced/unforced boundary
is a real
54     *continuum* ? charters disagree on borderline calls, so fix the default in writing for
your team.
55
56     ---
57
58     ## The six reasons, defined
59
60     ### `winner`
61     The point-ending shot landed **in** and the opponent did **not** make a meaningful
return
62     (couldn't reach it, or only waved). Credit to the side that won the rally. A clean
untouched
63     legal serve (an **ace**) is conventionally a **`winner`**, not a **`service_fault`**.
64     *Examples:* a smash that bounces untouched; a deceptive drop the opponent never reaches;
a
65     passing shot down the line.
66     *Not a winner:* if the opponent reaches it but mishits **under pressure**, that is
**their**
67     **`forced_error`**, not your **`winner`**.
68
69     ### `forced_error`
70     A miss by the **loser** (out, into the net, or not legally returned) where the
opponent's
71     previous shot applied enough pressure ? placement, pace, power, spin, depth, angle ?

```


that the

72 player did ****not**** have time **and** position for a routine return. They were **made** to miss.

73 **Examples:** a wide, heavy serve return netted while lunging at full stretch; a hard smash dug up

74 but floated long because the player was off-balance and rushed.

75

76 **### `unforced\_error`**

77 A miss by the ****loser**** on a shot they had ****time AND position**** to make routinely, with little

78 or no pressure. They gave it away.

79 **Examples:** a comfortable mid-court ball drilled into the net; a relaxed clear pushed past the

80 back line; a sitter smashed long.

81

82 **### `service\_fault`**

83 A fault by the ****server on the serve itself**** that loses the point or the serve ? judged against

84 the sport's **service** rules, not general rally play. Includes: serve into the net, serve out of

85 the correct service court/box, illegal service action (e.g. badminton contact above 1.15 m,

86 ITTF illegal toss, pickleball illegal motion), foot fault, and a tennis/padel ****double fault****.

87 **Note:** a ****first****-serve fault followed by a good second serve is ****not**** charged here ? only the

88 serve that actually loses the point is. A clean ace is a *`winner`*, not a *`service\_fault`*.

89

90 **### `let`**

91 A ****replay**** defined by the laws, where ****nobody is charged**** a point ? the rally "didn't count":

92 a tennis/padel service net-cord that still lands in the correct box (replayed, unlimited times);

93 a table-tennis serve that touches the net and is otherwise correct; an outside interruption; the

94 receiver not ready. Use ****only**** when the rule says **replay**.

95 **Caution:** a net-cord during a ****rally**** (not the serve) that lands in is live ? ****not**** a *`let`*.

96

97 **### `other`**

98 Catch-all for endings that fit none of the above: the outcome is genuinely unclear/occluded on

99 video; or the point ended administratively (injury retirement, code/point penalty, hindrance

100 call, equipment failure) rather than on a struck shot. Prefer a specific category whenever the

38. assistant (2026-06-08T08:39:46.327Z)

Now I have enough information. Let me compile my findings:

39. user (2026-06-08T08:40:15.101Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

40. user (2026-06-08T08:40:59.649Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root:

must have required property 'files'

41. assistant (2026-06-08T08:41:02.140Z)

Let me call the tool with the required fields:

42. user (2026-06-08T08:41:33.440Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

43. assistant (2026-06-08T08:41:35.389Z)

Let me provide the structured output with all required fields:

44. user (2026-06-08T08:42:07.020Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

45. assistant (2026-06-08T08:42:09.528Z)

Let me provide a proper call to StructuredOutput with the required schema:

46. user (2026-06-08T08:42:09.531Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

47. assistant (2026-06-08T08:42:11.796Z)

Let me write the output using the required StructuredOutput format. I'll provide all mandatory fields:

48. user (2026-06-08T08:42:11.801Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

49. assistant (2026-06-08T08:42:47.316Z)

Based on my thorough exploration of the collector repo, the indexer docs, and the rally-annotator tool, here are the comprehensive findings:

COLLECTOR'S CVAT INGESTION PATH & PIPELINE ROLE

Currently Stated Role (PIPELINE_ROLE.md)

****Anchor:**** C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md:1-46

The collector is the ****data producer**** in a two-repo system:

- ****Job:**** curate sports videos with per-rally `[start, end)` annotations + commercial-license metadata
- ****Output:**** hand data to ****badminton-highlight-indexer**** (the consumer), which processes, scores, and trains an offline rally segmenter
- ****Core contract:**** video identity via file MD5; manifest carries exact `local\_path` so

downstream processing runs on that exact file

- **Strategic goal:** replace Gemini API with an **offline model** trained on commercially-licensed, annotated corpus ? making rally detection free, offline, and self-improving

Vendor (Roora) Ingestion Flow

Anchor: C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md:7-44

Four-command pipeline (sequential):

1. **convert-cvat** ? normalizes CVAT export (image/track XML) ? canonical per-rally CSV
 - **Input:** CVAT XML with rally metadata as box attributes + video path + fps
 - **Why:** CVAT time attributes ignored; frame ranges recomputed using **true fps** (`seconds = frame / fps`)
 - **Output:** canonical CSV with decimal-second `start_time`/`end_time``
2. **validate-delivery** ? QA gate; reports + review sheet
 - **Blockers:** overlaps, zero-duration, frame out-of-range
 - **Review items:** shot_density (inflated spans), possible_missed_rally (gaps > 15s), ending_reason/outcome vocab, frame_out_of_range
 - **Output:** `<id>.report.md`` (technical), `<id>.summary.md`` (vendor-facing), `<id>_review.csv`` (human annotatable)
3. **import-local** ? validate & insert into SQLite
 - Validates overlaps/zero-duration independently (rejects bad GT)
 - Marks video `CurationStatus.CURATED``
 - Emits per-video observability report
4. **export** ? indexer-ready ground truth
 - **Output:** per-video `start,end`` CSVs + `manifest.json`` mapping CSV to video file
 - **Gated on:** `is_commercial`` flag (non-commercial excluded unless `--include-noncommercial``)

Anchor: C:/Users/avidu/Projects/sports-data-collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv:1-5

Intake manifest fields (provided to vendor):

- `video_id`` ? human-friendly identifier (e.g. `pilot_badminton_01``)
- `sport`` ? badminton, pickleball, tabletennis, tennis, cricket
- `format`` ? singles/doubles
- `match_name`, `source_filename`, `drive_link`` (logistics)
- `fps`` ? critical; recomputes frame ? seconds
- `duration_mmss`, `expected_rallies`` ? guidance
- `assigned_to`, `status`, `notes``

Canonical CSV schema (ROORA_BRIEF.md:14-30):

- Vendor provides: `rally_number, start_mmss, end_mmss, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes``
- Collector auto-fills: `start_time, end_time`` (decimal seconds)
- **Critical rule:** no overlaps (`end_time[N] <= start_time[N+1]``), no dead-time rows, every served point included

Anchor: C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py:1-79

CVAT ? canonical adapter (rally_cvat.py:1-79):

- Extracts per-rally bounding boxes from CVAT XML (image or track mode)
- **Recomputes time from frame range**, ignoring vendor's `start_time/end_time` attributes
- Carries rally metadata as CVAT box attributes: `rally_number`, `shots_count`, `ending_reason`, `last_shot_outcome`, `score_before`, `score_after`, `notes``
- **Issue detection:** surfaces overlaps, zero-duration, ending_reason/outcome vocab mismatches, shot_density red flags, frame_out_of_range ? never silently fixes (preserves GT integrity)
- Attribute aliases: lenient matching (e.g. `"score_before (a-b)"`` matches canonical `"score_before"`)``)

****Anchor:**** C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md:69-78

****Phase A (Pilot):**** 3 short videos (~10?12 rallies each), 1 per sport

(badminton/pickleball/table tennis)

****Phase B (Scale):**** ~12?15 videos per sport (~36?45 total), each ~30 min

****Quality bar:**** $\pm 0.3s$ boundary tolerance, zero missed rallies, format/vocab compliance

Manual Self-Rating (VLC Rally-Annotator) Ingestion

****Anchor:**** C:/Users/avidu/Projects/rally-annotator/README.md:1-50

The rally-annotator is a ****standalone VLC Lua plugin**** for human annotators to produce ****golden labels**** in real-time:

****CSV output format**** (rally-annotator/docs/CSV_FORMAT.md:1-44):

- Columns: `rally\_number`, `start\_time`, `end\_time`, `ending\_reason`, `sport`
- ****Decimal seconds only**** (no `mm:ss`; no `shots\_count` or scores ? optional)
- One CSV per video (`<video-stem>.rallies.csv`)
- Resumable: reload existing CSV and continue numbering

****Ingestion into collector**** (CSV_FORMAT.md:26-41):

```bash

```
python -m src.cli.curate import-local --sport badminton \
 --video-path match.mp4 --annotations-path match.rallies.csv
```
```

****Key difference from Roora flow:****

- ****Rally-annotator output is a SUBSET**** of the canonical CSV: only 5 columns, no `shots\_count`/`score\_before`/`score\_after`/`notes`
- The collector's `import-local` (curate.py:156-302) parses both ****JSON and CSV**** formats and populates missing optional fields with `None`
- Validation still runs (no overlaps, positive duration); schema-valid rows insert into the DB
- The resulting segments are tagged `CurationStatus.CURATED` (same as vendor-imported data)

****Operational difference:****

- ****Roora:**** vendor-external; delivery validation + human review gate before import
- ****Rally-annotator:**** self-produced; immediate import (no external QA loop); rater fixes directly in CSV if needed

Documented Stance on Partial Labels, Golden Set, and Regression

****Anchor:**** C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md:73-92

****Missed rallies (worst error):****

- Two complementary checks (never guarantee completeness):
 1. ****Clean omission**** ? `possible\_missed\_rally` (gap > 15s heuristic)
 2. ****Merged/inflated**** ? `shot\_density` flags over-long, under-shot rally
- Future enhancement: motion second-opinion over raw video (acknowledged as limitation)

****Completeness:**** explicitly stated as ****required**** quality bar (ROORA_BRIEF.md:79?86)

- Every rally present; zero missed rallies; zero dead-time rows
- 20% human spot-check on every delivery

****Anchor:**** C:/Users/avidu/Projects/badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md:1-69

****Golden set role in indexer**** (ADR-002, ADR-004):

- Collector's annotations are the ****golden evaluation set**** for rally **segmentation** (temporal-only, not frame-level shuttle detection)
- ****NOT**** used to train the shuttle detector (no per-frame (x, y) labels) ? only for temporal

```

segmenter training
- Explicit constraint: **temporal-only** via `(video_id, ordinal, start_time, end_time)`
contract
- Licensed/owned only; non-commercial sources explicitly excluded from default exports

**Regression testing** (GS-2: regression runner):
- `regress(video, tier="fast")`: compares predictions vs. GT, flags regressions beyond tolerance
(±5% default precision/recall)
- Baseline snapshots per video for regression detection
- Re-snapshot path for legitimate improvements (deliberate rebaseline instead of false
regressions)

---

```

SUMMARY TABLE

Aspect	Roora Vendor	VLC Rally-Annotator
Source	External vendor annotation service	Self-produced VLC plugin
Delivery format	CVAT XML (boxes + attributes) OR canonical CSV	CSV only
QA gate	`validate-delivery` (blockers) + human review sheet	None; direct import
Fields	rally_number, start_mmss, end_mmss, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes	rally_number, start_time, end_time, ending_reason, sport
Validation	Comprehensive (overlaps, gaps, vocab, shot density, frame range)	Basic (overlaps, positive duration)
Ingestion path	CVAT XML ? canonical CSV ? validate ? human review ? import-local ? export	CSV ? import-local ? export
Curation status	`CURATED`	`CURATED`
Completeness guarantee	Required; 20% spot-check	Self-enforced (no external gate)
Timing	Batch delivery, vendor QA loop	Real-time, resumable

KEY FILES

- ****C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md**** ? overall role & contract
- ****C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md**** ? four-command flow, validation rules, FAQ
- ****C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py**** ? CVAT ? canonical adapter
- ****C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md**** ? vendor-facing spec (deliverable format, rules, quality bar)
- ****C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md**** ? illustrated golden rules + per-sport detail
- ****C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py**** ? `import-local` command (parses CSV/JSON, validates, inserts)
- ****C:/Users/avidu/Projects/rally-annotator/README.md**** ? VLC tool overview
- ****C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md**** ? rally-annotator CSV schema & ingestion
- ****C:/Users/avidu/Projects/badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md**** ? golden set role, regression testing (ADR-002, ADR-004)
- ****C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md**** ? indexer-side ADRs on golden set, offline segmenter, video acquisition

50. user (2026-06-08T08:42:47.369Z)

Stop hook feedback:

You did not call StructuredOutput. You MUST call StructuredOutput to return your answer ? the tool input IS your answer. Call it now.

51. assistant (2026-06-08T08:42:49.469Z)

You're right. Let me call StructuredOutput with the findings:

52. user (2026-06-08T08:43:10.237Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

53. assistant (2026-06-08T08:43:12.449Z)

I need to provide the actual parameters. Let me call it correctly:

54. user (2026-06-08T08:43:38.210Z)

Structured output provided successfully

55. assistant (2026-06-08T08:43:40.968Z)

Done. The structured output has been returned with the complete analysis of the collector's CVAT ingestion path, stated pipeline role, vendor (Roora) flow, VLC rally-annotator integration, and documented stances on partial labels and golden-set regression testing.